

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Rhythm Trainer</title>
  <style>
    /* Bravura: SMuFL reference font (SIL OFL 1.1) — https://github.com/steinbergmedia/bravura
    */
    @font-face {
      font-family: Bravura;
      src:
url("https://raw.githubusercontent.com/steinbergmedia/bravura/master/redis/woff/Bravura.woff")
format("woff");
      font-display: block;
    }
    * {
      box-sizing: border-box;
    }
    html, body {
      height: 100%;
      margin: 0;
    }
    body {
      background: #000000;
      color: #e8e8e8;
      font-family: system-ui, sans-serif;
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
      min-height: 100%;
      padding: 1.5rem;
    }
    body.has-started {
      justify-content: flex-start;
      padding-top: 2.5rem;
    }
    .landing-title {
      margin: 0 0 1rem;
      color: #ffffff;
      font-size: 1.4rem;
      letter-spacing: 0.08em;
      font-weight: 700;

```

```
text-transform: uppercase;
text-align: center;
transform: translateY(-10rem);
}
.landing-subtitle {
margin: -0.6rem 0 1rem;
color: #ffffff;
font-size: 0.9rem;
letter-spacing: 0.04em;
text-align: center;
transform: translateY(-10rem);
}
body.has-started .landing-title {
display: none;
}
body.has-started .landing-subtitle {
display: none;
}
#start {
font-size: 1.25rem;
font-weight: 600;
padding: 0.75rem 2.5rem;
border: none;
border-radius: 8px;
cursor: pointer;
background: #22c55e;
color: #052e14;
}
#start:hover {
filter: brightness(1.08);
}
#start:active {
filter: brightness(0.95);
}
#practice-panel {
width: min(100%, 40rem);
display: flex;
flex-direction: column;
gap: 1rem;
}
#practice-panel[hidden] {
display: none !important;
}
.instructions {
```

```
margin: 0;
line-height: 1.5;
font-size: 1rem;
}
.rhythm-scroll {
width: 100%;
overflow-x: auto;
overflow-y: hidden;
border: 1px solid #333;
border-radius: 6px;
background: #0a0a0a;
}
.rhythm-scroll-inner {
display: flex;
align-items: flex-end;
gap: 1rem;
min-width: 200%;
min-height: 4.75rem;
padding: 0.35rem 1.25rem 0.5rem;
color: #e8e8e8;
}
.rhythm-scroll-inner svg {
cursor: pointer;
}
#rhythm-session {
width: min(100%, 40rem);
display: flex;
flex-direction: column;
align-items: center;
gap: 1rem;
}
#rhythm-session[hidden] {
display: none !important;
}
#rhythm-session.exercises-mode .detail-transport {
display: none !important;
}
#rhythm-session.know-all-detail .detail-transport {
display: none !important;
}
#rhythm-session.know-all-detail #quarter-tone-row {
display: none !important;
}
#rhythm-session-chrome {
```

```
width: 100%;
display: flex;
flex-direction: column;
align-items: center;
gap: 0.25rem;
}
#rhythm-detail {
width: 100%;
min-height: 50vh;
display: flex;
flex-direction: column;
align-items: center;
}
#rhythm-detail[hidden] {
display: none !important;
}
.detail-toolbar {
width: 100%;
display: flex;
flex-direction: row;
align-items: center;
justify-content: space-between;
gap: 1rem;
padding: 0 0.25rem;
margin-bottom: 0.25rem;
}
.metronome-control {
display: flex;
flex-direction: row;
align-items: center;
gap: 0.35rem;
}
.metronome-control button {
width: 2rem;
height: 2rem;
padding: 0;
border: 1px solid #334155;
border-radius: 6px;
background: #111827;
color: #e8e8e8;
font-size: 1.25rem;
line-height: 1;
cursor: pointer;
}
```

```
.metronome-control button:hover {
  background: #1e293b;
}
#bpm-value {
  min-width: 2.75rem;
  text-align: center;
  font-size: 1.5rem;
  font-weight: 700;
  color: #3b82f6;
  font-variant-numeric: tabular-nums;
}
.flash-bar {
  width: 14px;
  height: 52px;
  border-radius: 6px;
  background: #1e293b;
  border: 1px solid #334155;
  transition: background-color 45ms ease-out, box-shadow 45ms ease-out;
}
#bpm-flash-bar.flash {
  background: #3b82f6;
  box-shadow: 0 0 14px rgba(59, 130, 246, 0.65);
}
#rhythm-flash-bar.flash {
  background: #facc15;
  box-shadow: 0 0 14px rgba(250, 204, 21, 0.65);
}
#rhythm-flash-bar.sustain {
  animation: rhythmYellowHold var(--beat-duration, 500ms) ease-out forwards;
}
@keyframes rhythmYellowHold {
  0% {
    background: #facc15;
    box-shadow: 0 0 16px rgba(250, 204, 21, 0.75);
  }
  50% {
    background: #facc15;
    box-shadow: 0 0 16px rgba(250, 204, 21, 0.75);
  }
  99% {
    background: #facc15;
    box-shadow: 0 0 0 rgba(250, 204, 21, 0);
  }
  100% {
```

```
    background: #1e293b;
    box-shadow: none;
  }
}
#detail-title {
  margin: 0 0 0.5rem;
  color: #ffffff;
  font-size: 1.6rem;
  font-weight: 700;
  text-align: center;
  letter-spacing: 0.02em;
}
#detail-title[hidden] {
  display: none;
}
#detail-symbol {
  width: 100%;
  display: flex;
  justify-content: center;
  margin-top: 0.5rem;
  min-height: 7rem;
}
#detail-symbol svg {
  display: block;
  color: #e8e8e8;
}
#detail-text {
  margin: auto 0;
  color: #ffffff;
  font-size: 1.35rem;
  text-align: center;
  white-space: pre-line;
}
.detail-transport {
  display: flex;
  flex-direction: row;
  align-items: center;
  justify-content: center;
  gap: 1rem;
  margin: 0.75rem 0 0;
}
#detail-play {
  width: 0;
  height: 0;
```

```
border-top: 22px solid transparent;
border-bottom: 22px solid transparent;
border-left: 36px solid #22c55e;
background: transparent;
padding: 0;
cursor: pointer;
}
#detail-play[hidden] {
  display: none !important;
}
#detail-play:hover {
  filter: brightness(1.08);
}
#detail-play:active {
  filter: brightness(0.95);
}
#detail-stop {
  width: 28px;
  height: 28px;
  padding: 0;
  border: none;
  border-radius: 4px;
  background: #dc2626;
  cursor: pointer;
  flex-shrink: 0;
}
#detail-stop:hover {
  filter: brightness(1.08);
}
#detail-stop:active {
  filter: brightness(0.95);
}
#quarter-tone-row {
  width: 100%;
  max-width: 22rem;
  margin: 0 auto 0.75rem;
  display: none;
  flex-direction: column;
  align-items: stretch;
  gap: 0.35rem;
  font-size: 0.9rem;
  color: #cbd5e1;
}
#quarter-tone-row.visible {
```

```
    display: flex;
  }
  #quarter-tone-row label {
    display: flex;
    flex-direction: row;
    align-items: center;
    justify-content: space-between;
    gap: 0.75rem;
  }
  #quarter-freq {
    flex: 1;
    min-width: 0;
  }
  #quarter-freq-value {
    font-variant-numeric: tabular-nums;
    color: #e8e8e8;
    min-width: 3.25rem;
    text-align: right;
  }
  #detail-second-text {
    margin: 0;
    color: #ffffff;
    font-size: 1.1rem;
    text-align: center;
    white-space: pre-line;
  }
  #detail-second-text[hidden] {
    display: none !important;
  }
  #rhythm-session.know-all-detail #rhythm-session-chrome {
    display: none !important;
  }
  #rhythm-detail.know-all-detail #detail-symbol {
    display: none !important;
  }
  #rhythm-detail.know-all-detail #detail-text {
    margin: auto;
    font-size: 1.55rem;
  }
  .know-all-btn {
    flex-shrink: 0;
    align-self: flex-end;
    margin-bottom: 0.45rem;
    max-width: 12rem;
```



```

padding: 0.5rem 0.7rem;
font-size: 0.78rem;
line-height: 1.3;
font-weight: 600;
color: #fef9c3;
background: #3f2e0a;
border: 1px solid #ca8a04;
border-radius: 8px;
cursor: pointer;
text-align: center;
}
.know-all-btn:hover {
  filter: brightness(1.12);
}
.know-all-btn:active {
  filter: brightness(0.92);
}
.quarter-note,
.quarter-rest,
.eighth-beam-pair,
.eighth-note,
.eighth-rest,
.dotted-eighth,
.sixteenth-beam-pair,
.sixteenth-note,
.sixteenth-rest,
.rhythm-eighth-two-sixteenths,
.rhythm-two-sixteenths-eighth,
.rhythm-dotted-eighth-sixteenth,
.rhythm-sixteenth-dotted-eighth,
.rhythm-syncopation {
  flex-shrink: 0;
  display: block;
}
.quarter-rest .quarter-rest-glyph,
.eighth-rest .eighth-rest-glyph,
.sixteenth-rest .sixteenth-rest-glyph {
  font-family: Bravura, serif;
  fill: currentColor;
}
#detail-next-wrap {
  margin-top: auto;
  width: 100%;
  padding-top: 1.25rem;

```

```
display: flex;
flex-direction: column;
align-items: center;
gap: 0.35rem;
}
#rhythm-detail.know-all-detail #detail-title {
margin-top: 0;
}
#detail-next {
font-size: 1.05rem;
font-weight: 600;
padding: 0.65rem 1.75rem;
border: none;
border-radius: 8px;
cursor: pointer;
background: #2563eb;
color: #eff6ff;
}
#detail-next:hover {
filter: brightness(1.08);
}
#detail-next:active {
filter: brightness(0.95);
}
#rhythm-trainer-mode-buttons {
display: flex;
flex-direction: row;
gap: 0.75rem;
flex-wrap: wrap;
justify-content: center;
}
#rhythm-trainer-mode-buttons[hidden] {
display: none !important;
}
.trainer-mode-btn {
font-size: 1.05rem;
font-weight: 600;
padding: 0.65rem 1.5rem;
border: none;
border-radius: 8px;
cursor: pointer;
background: #2563eb;
color: #eff6ff;
min-width: 9rem;
```

```

}
.trainer-mode-btn.challenging {
  background: #b91c1c;
  color: #fef2f2;
}
.trainer-mode-btn:hover {
  filter: brightness(1.1);
}
.trainer-mode-btn:active {
  filter: brightness(0.92);
}
#exercises-panel {
  width: 100%;
  display: flex;
  flex-direction: column;
  gap: 1rem;
  padding-bottom: 2rem;
}
#exercises-panel[hidden] {
  display: none !important;
}
#exercises-panel h2 {
  margin: 0;
  color: #ffffff;
  font-size: 1.35rem;
}
#exercise-play-sequence,
#exercise-check,
#exercise-next {
  font-size: 1rem;
  font-weight: 600;
  padding: 0.55rem 1.25rem;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  align-self: flex-start;
}
#exercise-play-sequence {
  background: #22c55e;
  color: #052e14;
}
#exercise-check {
  background: #7c3aed;
  color: #f5f3ff;
}

```

```
}
#exercise-next {
  background: #2563eb;
  color: #eff6ff;
}
#exercise-play-sequence:hover,
#exercise-check:hover,
#exercise-next:hover {
  filter: brightness(1.08);
}
#exercise-next[hidden] {
  display: none !important;
}
#exercise-slots {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 0.75rem;
}
#exercise-slots.slots-two {
  grid-template-columns: repeat(2, 1fr);
}
@media (min-width: 520px) {
  #exercise-slots:not(.slots-two) {
    grid-template-columns: repeat(4, 1fr);
  }
}
.exercise-slot {
  display: flex;
  flex-direction: column;
  gap: 0.35rem;
}
.slot-label {
  font-size: 0.8rem;
  color: #94a3b8;
  text-transform: uppercase;
  letter-spacing: 0.06em;
}
.drop-zone {
  min-height: 5.5rem;
  border: 2px dashed #475569;
  border-radius: 8px;
  background: #0f172a;
  display: flex;
  align-items: center;
```

```
    justify-content: center;
    padding: 0.35rem;
    transition: border-color 0.15s, background 0.15s;
}
.drop-zone.drop-hover {
    border-color: #38bdf8;
    background: #172554;
}
.drop-zone.filled {
    border-style: solid;
    border-color: #334155;
}
.drop-zone svg {
    display: block;
    color: #e8e8e8;
    max-width: 100%;
    height: auto;
}
.exercise-bank-label {
    margin: 0.5rem 0 0;
    font-size: 0.9rem;
    color: #94a3b8;
}
.exercise-bank-label[hidden] {
    display: none !important;
}
#exercise-answer-area[hidden] {
    display: none !important;
}
.exercise-bank {
    display: flex;
    flex-wrap: wrap;
    gap: 0.65rem;
    align-items: flex-end;
    padding: 0.75rem;
    border: 1px solid #333;
    border-radius: 8px;
    background: #0a0a0a;
    min-height: 5rem;
}
.rhythm-drag {
    cursor: grab;
    padding: 0.25rem 0.35rem;
    border-radius: 6px;
```

```
background: #111827;
border: 1px solid #334155;
touch-action: none;
}
.rhythm-drag:active {
  cursor: grabbing;
}
.rhythm-drag svg {
  display: block;
  color: #e8e8e8;
  pointer-events: none;
}
.rhythm-drag.compound-chip {
  display: inline-flex;
  align-items: center;
  gap: 4px;
}
.rhythm-drag.compound-chip svg {
  display: inline-block;
}
#exercise-feedback {
  margin: 0;
  min-height: 1.5rem;
  font-size: 1rem;
  color: #fbbf24;
}
#exercise-back {
  font-size: 1rem;
  font-weight: 600;
  padding: 0.55rem 1.25rem;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  align-self: center;
  background: #334155;
  color: #e8e8e8;
}
#exercise-back:hover {
  filter: brightness(1.08);
}
.exercise-drag-block {
  padding-bottom: 0.25rem;
}
.exercise-group-fail {
```

```

width: 100%;
display: flex;
flex-direction: column;
align-items: center;
gap: 1.25rem;
padding: 1.5rem 1rem 2rem;
text-align: center;
}
.exercise-group-fail[hidden] {
display: none !important;
}
.exercise-group-fail-text {
margin: 0;
font-size: 1.25rem;
line-height: 1.45;
color: #fecaca;
max-width: 28rem;
}
.exercise-fail-next-btn {
font-size: 1rem;
font-weight: 600;
padding: 0.55rem 1.75rem;
border: none;
border-radius: 8px;
cursor: pointer;
background: #2563eb;
color: #eff6ff;
}
.exercise-fail-next-btn:hover {
filter: brightness(1.08);
}
</style>
</head>
<body>
<h1 class="landing-title">RHYTHM TRAINER</h1>
<p class="landing-subtitle">(and teacher)</p>
<button type="button" id="start">Start</button>
<div id="practice-panel" hidden>
<p class="instructions">
Pick the first rhythm from the left that you don't know (pick "I know all" if you know all of the
rhythms displayed).
</p>
<div class="rhythm-scroll" role="region" aria-label="Rhythm choices">
<div class="rhythm-scroll-inner">

```

```

<svg
  class="quarter-note"
  xmlns="http://www.w3.org/2000/svg"
  width="42"
  height="51"
  viewBox="0 -14 56 68"
  aria-label="Quarter note"
  role="img"
>
  <!-- Filled notehead (crotchet) + stem up -->
  <ellipse
    cx="22"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 22 38)"
  />
  <line
    x1="37"
    y1="-12"
    x2="37"
    y2="35"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="round"
  />
</svg>
<svg
  class="quarter-rest"
  xmlns="http://www.w3.org/2000/svg"
  width="42"
  height="51"
  viewBox="0 -14 56 68"
  aria-label="Quarter rest"
  role="img"
>
  <!-- SMuFL restQuarter (U+E4E5) via Bravura font — needs network once to load the
.woff -->
  <text
    class="quarter-rest-glyph"
    x="28"
    y="16"

```



```

    font-size="56"
    text-anchor="middle"
    dominant-baseline="central"
>&#xe4e5;</text>
</svg>
<svg
  class="eighth-beam-pair"
  xmlns="http://www.w3.org/2000/svg"
  width="78"
  height="51"
  viewBox="0 -14 104 68"
  aria-label="Two eighth notes beamed"
  role="img"
>
  <!-- Wider spacing (centers 22 / 70); same scale as quarter: 78/104 = 51/68 -->
  <ellipse
    cx="22"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 22 38)"
  />
  <ellipse
    cx="70"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 70 38)"
  />
  <!-- Beam spans stem stroke (width 2): centers 39 & 87 → x ∈ [38, 88] -->
  <rect
    x="36"
    y="-12"
    width="50"
    height="3"
    rx="0"
    fill="currentColor"
  />
  <line
    x1="37"

```

```

    y1="-9"
    x2="37"
    y2="35"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="butt"
  />
  <line
    x1="85"
    y1="-9"
    x2="85"
    y2="35"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="butt"
  />
</svg>
<svg
  class="eighth-note"
  xmlns="http://www.w3.org/2000/svg"
  width="42"
  height="51"
  viewBox="0 -14 56 68"
  aria-label="Eighth note"
  role="img"
>
  <!-- Quaver: filled head + stem + flag -->
  <ellipse
    cx="22"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 22 38)"
  />
  <line
    x1="37"
    y1="-12"
    x2="37"
    y2="35"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="round"

```

```

/>
<path
  d="M 39,-12 Q 55,-6 52,5"
  fill="none"
  stroke="currentColor"
  stroke-width="2"
  stroke-linecap="round"
/>
</svg>
<svg
  class="eighth-rest"
  xmlns="http://www.w3.org/2000/svg"
  width="42"
  height="51"
  viewBox="0 -14 56 68"
  overflow="visible"
  aria-label="Eighth rest"
  role="img"
>
  <!-- SMuFL rest8th (U+E4E6) via Bravura -->
  <text
    class="eighth-rest-glyph"
    x="28"
    y="22"
    font-size="72"
    text-anchor="middle"
    dominant-baseline="central"
  >&#xe4e6;</text>
</svg>
<!-- 6. Dotted eighth -->
<svg
  class="dotted-eighth"
  xmlns="http://www.w3.org/2000/svg"
  width="42"
  height="51"
  viewBox="0 -14 56 68"
  aria-label="Dotted eighth note"
  role="img"
>
  <ellipse
    cx="22"
    cy="38"
    rx="16"
    ry="9"

```

```

    fill="currentColor"
    stroke="none"
    transform="rotate(-16 22 38)"
  />
  <line
    x1="37"
    y1="-12"
    x2="37"
    y2="35"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="round"
  />
  <path
    d="M 39,-12 Q 55,-6 52,5"
    fill="none"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="round"
  />
  <circle cx="44" cy="38" r="3.6" fill="currentColor" />
</svg>
<!-- 7. Beamed sixteenths (four in one beat) -->
<svg
  class="sixteenth-beam-pair"
  xmlns="http://www.w3.org/2000/svg"
  width="114"
  height="51"
  viewBox="0 -14 152 68"
  aria-label="Four sixteenth notes beamed"
  role="img"
>
  <ellipse
    cx="20"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 20 38)"
  />
  <ellipse
    cx="54"
    cy="38"

```

```

    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 54 38)"
  />
  <ellipse
    cx="88"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 88 38)"
  />
  <ellipse
    cx="122"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 122 38)"
  />
  <rect x="34" y="-12" width="104" height="3" rx="0" fill="currentColor" />
  <rect x="34" y="-7" width="104" height="3" rx="0" fill="currentColor" />
  <line
    x1="35"
    y1="-12"
    x2="35"
    y2="35"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="butt"
  />
  <line
    x1="69"
    y1="-12"
    x2="69"
    y2="35"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="butt"
  />

```

```

<line
  x1="103"
  y1="-12"
  x2="103"
  y2="35"
  stroke="currentColor"
  stroke-width="2"
  stroke-linecap="butt"
/>
<line
  x1="137"
  y1="-12"
  x2="137"
  y2="35"
  stroke="currentColor"
  stroke-width="2"
  stroke-linecap="butt"
/>
</svg>
<!-- 8. Single sixteenth -->
<svg
  class="sixteenth-note"
  xmlns="http://www.w3.org/2000/svg"
  width="42"
  height="51"
  viewBox="0 -14 56 68"
  aria-label="Sixteenth note"
  role="img"
>
  <ellipse
    cx="22"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 22 38)"
  />
  <line
    x1="37"
    y1="-12"
    x2="37"
    y2="35"
    stroke="currentColor"

```

```

        stroke-width="2"
        stroke-linecap="round"
    />
    <path
        d="M 39,-12 Q 55,-6 52,5"
        fill="none"
        stroke="currentColor"
        stroke-width="2"
        stroke-linecap="round"
    />
    <path
        d="M 39,-6 Q 54,0 51,11"
        fill="none"
        stroke="currentColor"
        stroke-width="2"
        stroke-linecap="round"
    />
</svg>
<!-- 9. Sixteenth rest -->
<svg
    class="sixteenth-rest"
    xmlns="http://www.w3.org/2000/svg"
    width="42"
    height="51"
    viewBox="0 -14 56 68"
    overflow="visible"
    aria-label="Sixteenth rest"
    role="img"
>
    <text
        class="sixteenth-rest-glyph"
        x="28"
        y="22"
        font-size="72"
        text-anchor="middle"
        dominant-baseline="central"
    >&#xe4e7;</text>
</svg>
<!-- 10. Eighth followed by two sixteenths (double beam) -->
<svg
    class="rhythm-eighth-two-sixteenths"
    xmlns="http://www.w3.org/2000/svg"
    width="93"
    height="51"

```

```

viewBox="0 -14 124 68"
aria-label="Eighth note followed by two sixteenth notes beamed"
role="img"
>
<ellipse
  cx="22"
  cy="38"
  rx="16"
  ry="9"
  fill="currentColor"
  stroke="none"
  transform="rotate(-16 22 38)"
/>
<ellipse
  cx="66"
  cy="38"
  rx="16"
  ry="9"
  fill="currentColor"
  stroke="none"
  transform="rotate(-16 66 38)"
/>
<ellipse
  cx="102"
  cy="38"
  rx="16"
  ry="9"
  fill="currentColor"
  stroke="none"
  transform="rotate(-16 102 38)"
/>
<rect x="36" y="-12" width="82" height="3" rx="0" fill="currentColor" />
<!-- Lower beam only on the two sixteenths (right pair) -->
<rect x="80" y="-7" width="38" height="3" rx="0" fill="currentColor" />
<line
  x1="37"
  y1="-12"
  x2="37"
  y2="35"
  stroke="currentColor"
  stroke-width="2"
  stroke-linecap="butt"
/>
<line

```



```

x1="81"
y1="-12"
x2="81"
y2="35"
stroke="currentColor"
stroke-width="2"
stroke-linecap="butt"
/>
<line
x1="117"
y1="-12"
x2="117"
y2="35"
stroke="currentColor"
stroke-width="2"
stroke-linecap="butt"
/>
</svg>
<!-- 11. Two sixteenths followed by an eighth -->
<svg
class="rhythm-two-sixteenths-eighth"
xmlns="http://www.w3.org/2000/svg"
width="93"
height="51"
viewBox="0 -14 124 68"
aria-label="Two sixteenth notes followed by an eighth note beamed"
role="img"
>
<ellipse
cx="24"
cy="38"
rx="16"
ry="9"
fill="currentColor"
stroke="none"
transform="rotate(-16 24 38)"
/>
<ellipse
cx="56"
cy="38"
rx="16"
ry="9"
fill="currentColor"
stroke="none"

```

```

    transform="rotate(-16 56 38)"
  />
  <ellipse
    cx="96"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 96 38)"
  />
  <rect x="38" y="-12" width="74" height="3" rx="0" fill="currentColor" />
  <!-- Lower beam only on the two sixteenths (left pair) -->
  <rect x="38" y="-7" width="34" height="3" rx="0" fill="currentColor" />
  <line
    x1="39"
    y1="-12"
    x2="39"
    y2="35"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="butt"
  />
  <line
    x1="71"
    y1="-12"
    x2="71"
    y2="35"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="butt"
  />
  <line
    x1="111"
    y1="-12"
    x2="111"
    y2="35"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="butt"
  />
</svg>
<!-- 12. Dotted eighth + sixteenth (beamed) -->
<svg

```

```

class="rhythm-dotted-eighth-sixteenth"
xmlns="http://www.w3.org/2000/svg"
width="67"
height="51"
viewBox="0 -14 90 68"
aria-label="Dotted eighth and sixteenth notes beamed"
role="img"
>
<ellipse
  cx="20"
  cy="38"
  rx="16"
  ry="9"
  fill="currentColor"
  stroke="none"
  transform="rotate(-16 20 38)"
/>
<circle cx="44" cy="38" r="3.6" fill="currentColor" />
<ellipse
  cx="66"
  cy="38"
  rx="16"
  ry="9"
  fill="currentColor"
  stroke="none"
  transform="rotate(-16 66 38)"
/>
<rect x="34" y="-12" width="48" height="3" rx="0" fill="currentColor" />
<rect x="34" y="-7" width="48" height="3" rx="0" fill="currentColor" />
<line
  x1="35"
  y1="-12"
  x2="35"
  y2="35"
  stroke="currentColor"
  stroke-width="2"
  stroke-linecap="butt"
/>
<line
  x1="81"
  y1="-12"
  x2="81"
  y2="35"
  stroke="currentColor"

```

```

        stroke-width="2"
        stroke-linecap="butt"
    />
</svg>
<!-- 13. Sixteenth + dotted eighth (beamed) -->
<svg
  class="rhythm-sixteenth-dotted-eighth"
  xmlns="http://www.w3.org/2000/svg"
  width="67"
  height="51"
  viewBox="0 -14 90 68"
  aria-label="Sixteenth and dotted eighth notes beamed"
  role="img"
>
  <ellipse
    cx="20"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 20 38)"
  />
  <ellipse
    cx="58"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 58 38)"
  />
  <circle cx="78" cy="38" r="3.6" fill="currentColor" />
  <rect x="34" y="-12" width="40" height="3" rx="0" fill="currentColor" />
  <rect x="34" y="-7" width="40" height="3" rx="0" fill="currentColor" />
  <line
    x1="35"
    y1="-12"
    x2="35"
    y2="35"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="butt"
  />

```

```

<line
  x1="73"
  y1="-12"
  x2="73"
  y2="35"
  stroke="currentColor"
  stroke-width="2"
  stroke-linecap="butt"
/>
</svg>
<!-- 14. Syncopation: sixteenth, eighth, sixteenth (beamed) -->
<svg
  class="rhythm-syncopation"
  xmlns="http://www.w3.org/2000/svg"
  width="93"
  height="51"
  viewBox="0 -14 124 68"
  aria-label="Syncopation: sixteenth, eighth, and sixteenth notes beamed"
  role="img"
>
  <ellipse
    cx="20"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 20 38)"
  />
  <ellipse
    cx="54"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"
    stroke="none"
    transform="rotate(-16 54 38)"
  />
  <ellipse
    cx="96"
    cy="38"
    rx="16"
    ry="9"
    fill="currentColor"

```

```

        stroke="none"
        transform="rotate(-16 96 38)"
    />
    <rect x="34" y="-12" width="78" height="3" rx="0" fill="currentColor" />
    <!-- Broken lower beam: short flag from first sixteenth and short flag from last sixteenth -->
    <rect x="34" y="-7" width="15" height="3" rx="0" fill="currentColor" />
    <rect x="97" y="-7" width="15" height="3" rx="0" fill="currentColor" />
    <line
        x1="35"
        y1="-12"
        x2="35"
        y2="35"
        stroke="currentColor"
        stroke-width="2"
        stroke-linecap="butt"
    />
    <line
        x1="69"
        y1="-12"
        x2="69"
        y2="35"
        stroke="currentColor"
        stroke-width="2"
        stroke-linecap="butt"
    />
    <line
        x1="111"
        y1="-12"
        x2="111"
        y2="35"
        stroke="currentColor"
        stroke-width="2"
        stroke-linecap="butt"
    />
</svg>
<button type="button" class="know-all-btn" id="know-all-rhythms">
    I already know all of them!
</button>
</div>
</div>
</div>
<div id="rhythm-session" hidden>
    <div id="rhythm-session-chrome">
        <div class="detail-toolbar">

```

```

<div id="bpm-flash-bar" class="flash-bar" aria-hidden="true"></div>
<div class="metronome-control" role="group" aria-label="Metronome tempo">
  <button type="button" id="bpm-minus" aria-label="Slower">--</button>
  <span id="bpm-value">80</span>
  <button type="button" id="bpm-plus" aria-label="Faster">+</button>
</div>
<div id="rhythm-flash-bar" class="flash-bar" aria-hidden="true"></div>
</div>
<div id="quarter-tone-row" aria-hidden="true">
  <label for="quarter-freq">
    <span>Note pitch (Hz)</span>
    <span id="quarter-freq-value">440</span>
  </label>
  <input id="quarter-freq" type="range" min="110" max="880" step="1" value="440" />
</div>
<div class="detail-transport">
  <button id="detail-play" type="button" aria-label="Play"></button>
  <button id="detail-stop" type="button" aria-label="Stop"></button>
</div>
</div>
<div id="rhythm-detail" hidden>
  <h2 id="detail-title" aria-live="polite"></h2>
  <div id="detail-symbol"></div>
  <p id="detail-text">(replacement text)</p>
  <p id="detail-second-text" hidden></p>
  <div id="detail-next-wrap">
    <button type="button" id="detail-next" aria-label="Next: exercises" hidden>
      Next
    </button>
  <div id="rhythm-trainer-mode-buttons" hidden>
    <button
      type="button"
      id="trainer-mode-original"
      class="trainer-mode-btn"
      aria-label="Start rhythm trainer in Original mode"
    >
      Original
    </button>
    <button
      type="button"
      id="trainer-mode-challenging"
      class="trainer-mode-btn challenging"
      aria-label="Start rhythm trainer in Challenging mode"
    >

```

```

        Challenging
    </button>
</div>
</div>
</div>
<div id="exercises-panel" hidden>
<h2>Exercises</h2>
<div id="exercise-main-block" class="exercise-drag-block">
    <button type="button" id="exercise-play-sequence">Play exercises</button>
    <div id="exercise-answer-area">
        <div id="exercise-slots" role="group" aria-label="Answer boxes"></div>
        <p id="exercise-bank-label" class="exercise-bank-label" hidden></p>
        <div id="exercise-bank" class="exercise-bank"></div>
        <button type="button" id="exercise-check">Check answers</button>
    </div>
    <p id="exercise-feedback"></p>
    <button type="button" id="exercise-next" hidden aria-label="Continue">
        Next
    </button>
</div>
<div id="exercise-group-fail" class="exercise-group-fail" hidden>
    <p class="exercise-group-fail-text">
        You failed this group of exercises; you should try again!
    </p>
    <button
        type="button"
        id="exercise-fail-next"
        class="exercise-fail-next-btn"
        aria-label="Return to the same rhythm lesson"
    >
        Next
    </button>
</div>
<button type="button" id="exercise-back">Back to rhythms</button>
</div>
</div>
<script>
const startButton = document.getElementById("start");
const practicePanel = document.getElementById("practice-panel");
const rhythmSession = document.getElementById("rhythm-session");
const detailPanel = document.getElementById("rhythm-detail");
const detailTitle = document.getElementById("detail-title");
const detailSymbol = document.getElementById("detail-symbol");

```



```

const RHYTHM_DISPLAY_NAMES = {
  "quarter-note": "Quarter note",
  "quarter-rest": "Quarter rest",
  "eighth-beam-pair": "Beamed eighth notes",
  "eighth-note": "Eighth note",
  "eighth-rest": "Eighth rest",
  "dotted-eighth": "Dotted eighth note",
  "sixteenth-beam-pair": "Beamed sixteenth notes",
  "sixteenth-note": "Sixteenth note",
  "sixteenth-rest": "Sixteenth rest",
  "rhythm-eighth-two-sixteenths": "Eighth note + two sixteenths",
  "rhythm-two-sixteenths-eighth": "Two sixteenths + eighth note",
  "rhythm-dotted-eighth-sixteenth": "Dotted eighth + sixteenth",
  "rhythm-sixteenth-dotted-eighth": "Sixteenth + dotted eighth",
  "rhythm-syncopation": "Syncopation",
  "rhythm-trainer": "Rhythm trainer"
};

const detailText = document.getElementById("detail-text");
const detailPlayButton = document.getElementById("detail-play");
const detailStopButton = document.getElementById("detail-stop");
const detailSecondText = document.getElementById("detail-second-text");
const quarterToneRow = document.getElementById("quarter-tone-row");
const quarterFreqInput = document.getElementById("quarter-freq");
const quarterFreqValueEl = document.getElementById("quarter-freq-value");
const bpmMinus = document.getElementById("bpm-minus");
const bpmPlus = document.getElementById("bpm-plus");
const bpmValueEl = document.getElementById("bpm-value");
const bpmFlashBar = document.getElementById("bpm-flash-bar");
const rhythmFlashBar = document.getElementById("rhythm-flash-bar");
const knowAllRhythmsBtn = document.getElementById("know-all-rhythms");
const exercisesPanel = document.getElementById("exercises-panel");
const detailNextBtn = document.getElementById("detail-next");
const exerciseBackBtn = document.getElementById("exercise-back");
const exercisePlayBtn = document.getElementById("exercise-play-sequence");
const exerciseCheckBtn = document.getElementById("exercise-check");
const exerciseBank = document.getElementById("exercise-bank");
const exerciseFeedback = document.getElementById("exercise-feedback");
const exerciseSlotsEl = document.getElementById("exercise-slots");
const exerciseBankLabelEl = document.getElementById("exercise-bank-label");
const exerciseAnswerArea = document.getElementById("exercise-answer-area");
const exerciseNextBtn = document.getElementById("exercise-next");
const exerciseMainBlockEl = document.getElementById("exercise-main-block");
const exerciseGroupFailEl = document.getElementById("exercise-group-fail");
const exerciseFailNextBtn = document.getElementById("exercise-fail-next");

```

```

const rhythmTrainerModeButtonsEl = document.getElementById(
  "rhythm-trainer-mode-buttons"
);
const trainerModeOriginalBtn = document.getElementById("trainer-mode-original");
const trainerModeChallengingBtn = document.getElementById(
  "trainer-mode-challenging"
);

/**
 * Rhythm trainer pools, organized into three difficulty sets. The trainer
 * draws each beat by first picking a set according to the active mode's
 * weights, then sampling uniformly within that set.
 */
const RHYTHM_TRAINER_SETS = {
  set1: [
    "quarter-note",
    "quarter-rest",
    "eighth-beam-pair",
    "compound-eighth-note-rest",
    "compound-eighth-rest-note"
  ],
  set2: [
    "rhythm-two-sixteenths-eighth",
    "compound-two-sixteenths-eighth-rest",
    "compound-sixteenth-rest-sixteenth-note-eighth-note",
    "compound-sixteenth-rest-sixteenth-note-eighth-rest",
    "compound-eighth-rest-sixteenth-rest-sixteenth-note",
    "rhythm-eighth-two-sixteenths",
    "compound-eighth-rest-two-sixteenths",
    "compound-sixteenth-note-eighth-rest-sixteenth-note",
    "rhythm-syncopation",
    "compound-sixteenth-rest-eighth-note-sixteenth-note",
    "rhythm-dotted-eighth-sixteenth",
    "rhythm-sixteenth-dotted-eighth",
    "compound-sixteenth-rest-dotted-eighth"
  ],
  set3: [
    "compound-sixteenth-note-sixteenth-rest-eighth-note",
    "compound-sixteenth-note-sixteenth-rest-eighth-rest",
    "compound-eighth-rest-sixteenth-note-sixteenth-rest",
    "compound-eighth-note-sixteenth-note-sixteenth-rest",
    "compound-eighth-note-sixteenth-rest-sixteenth-note",
    "compound-sixteenth-note-sixteenth-rest-sixteenth-note-sixteenth-rest",
    "compound-sixteenth-rest-two-sixteenths-sixteenth-rest",

```

```

    "compound-sixteenth-rest-sixteenth-note-sixteenth-rest-sixteenth-note",
    "compound-sixteenth-note-eighth-note-sixteenth-rest",
    "compound-dotted-eighth-sixteenth-rest",
    "compound-sixteenth-rest-eighth-note-sixteenth-rest"
  ]
};
const RHYTHM_TRAINER_MODES = {
  original: {
    label: "Original",
    sets: [
      { key: "set1", weight: 1 / 3 },
      { key: "set2", weight: 2 / 3 }
    ]
  },
  challenging: {
    label: "Challenging",
    sets: [
      { key: "set1", weight: 1 / 6 },
      { key: "set2", weight: 1 / 2 },
      { key: "set3", weight: 1 / 3 }
    ]
  }
};
let rhythmTrainerMode = "original";

function rhythmTrainerActiveSets() {
  const mode = RHYTHM_TRAINER_MODES[rhythmTrainerMode] ||
RHYTHM_TRAINER_MODES.original;
  return mode.sets;
}

function rhythmTrainerBankClasses() {
  const seen = {};
  const all = [];
  rhythmTrainerActiveSets().forEach(function (entry) {
    const setItems = RHYTHM_TRAINER_SETS[entry.key] || [];
    setItems.forEach(function (cls) {
      if (!seen[cls]) {
        seen[cls] = true;
        all.push(cls);
      }
    });
  });
  return all;
}

```

```

}

function drawRhythmTrainerBeat() {
  const buckets = rhythmTrainerActiveSets();
  let total = 0;
  buckets.forEach(function (b) {
    total += b.weight;
  });
  let r = Math.random() * total;
  let chosenSet = buckets[0];
  for (let i = 0; i < buckets.length; i += 1) {
    r -= buckets[i].weight;
    if (r <= 0) {
      chosenSet = buckets[i];
      break;
    }
  }
  const items = RHYTHM_TRAINER_SETS[chosenSet.key] || [];
  if (!items.length) {
    return null;
  }
  return items[Math.floor(Math.random() * items.length)];
}

/**
 * Per-rhythm drag-drill series. Each entry is:
 *   rhythmId: { exercises: [ { sequence, bankClasses, slotLabels, successMessage, usePitch,
playKind }, ... ] }
 *
 * Template for adding another drill on the same rhythm (before Next advances to the next
strip symbol):
 *   exercises: [
 *     { ... first drill ... },
 *     {
 *       sequence: ["quarter-note", "quarter-note", "quarter-note", "quarter-note"],
 *       bankClasses: ["quarter-note"],
 *       slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
 *       successMessage: "Yes — ...",
 *       usePitch: true,
 *       playKind: "per-beat"
 *     }
 *   ]
 *
 * After Check, Next is always available. A wrong first Check on a step disables

```

* "Play exercises" for that step until Next. On the last drill, Next leaves the group only if there was at

* least one Check and correct/total $\geq 80\%$; otherwise the failure screen appears.

*/

```
const EXERCISE_CONFIG = {
  "quarter-note": {
    exercises: [
      {
        sequence: [
          "quarter-note",
          "quarter-note",
          "quarter-note",
          "quarter-note"
        ],
        bankClasses: ["quarter-note"],
        slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
        successMessage: "Yes — all four beats are quarter notes.",
        usePitch: true,
        playKind: "per-beat"
      }
    ],
  },
  "quarter-rest": {
    exercises: [
      {
        sequence: [
          "quarter-note",
          "quarter-rest",
          "quarter-rest",
          "quarter-note"
        ],
        bankClasses: ["quarter-note", "quarter-rest"],
        slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
        successMessage:
          "Yes — quarter note, two quarter rests, then quarter note.",
        usePitch: true,
        playKind: "per-beat"
      },
      {
        sequence: [
          "quarter-note",
          "quarter-note",
          "quarter-note",
          "quarter-rest"
        ],
      }
    ],
  }
}
```

```

    ],
    bankClasses: ["quarter-note", "quarter-rest"],
    slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
    successMessage:
      "Yes — three quarter notes and a quarter rest on the last beat.",
    usePitch: true,
    playKind: "per-beat"
  }
]
},
"eighth-rest": {
  exercises: [
    {
      sequence: [
        "quarter-note",
        "compound-eighth-note-rest",
        "eighth-beam-pair",
        "quarter-note"
      ],
      bankClasses: [
        "quarter-note",
        "quarter-rest",
        "eighth-beam-pair",
        "compound-eighth-note-rest",
        "compound-eighth-rest-note"
      ],
      slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
      successMessage:
        "Yes — quarter note, eighth note + eighth rest, beamed eighths, quarter note.",
      usePitch: true,
      playKind: "per-beat"
    },
    {
      sequence: [
        "quarter-rest",
        "compound-eighth-rest-note",
        "compound-eighth-rest-note",
        "eighth-beam-pair"
      ],
      bankClasses: [
        "quarter-note",
        "quarter-rest",
        "eighth-beam-pair",
        "compound-eighth-note-rest",

```

```

    "compound-eighth-rest-note"
  ],
  slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
  successMessage:
    "Yes — quarter rest, two eighth rest + eighth note beats, then beamed eighths.",
  usePitch: true,
  playKind: "per-beat"
},
{
  sequence: [
    "eighth-beam-pair",
    "quarter-rest",
    "compound-eighth-note-rest",
    "quarter-note"
  ],
  bankClasses: [
    "quarter-note",
    "quarter-rest",
    "eighth-beam-pair",
    "compound-eighth-note-rest",
    "compound-eighth-rest-note"
  ],
  slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
  successMessage:
    "Yes — beamed eighths, quarter rest, eighth note + eighth rest, quarter note.",
  usePitch: true,
  playKind: "per-beat"
}
],
},
"eighth-beam-pair": {
  exercises: [
    {
      sequence: [
        "eighth-beam-pair",
        "quarter-note",
        "eighth-beam-pair",
        "quarter-note"
      ],
      bankClasses: ["quarter-note", "quarter-rest", "eighth-beam-pair"],
      slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
      successMessage:
        "Yes — beamed eighths and quarter notes alternating.",
      usePitch: true,
    }
  ]
}

```

```

    playKind: "per-beat"
  },
  {
    sequence: [
      "eighth-beam-pair",
      "eighth-beam-pair",
      "eighth-beam-pair",
      "quarter-note"
    ],
    bankClasses: ["quarter-note", "quarter-rest", "eighth-beam-pair"],
    slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
    successMessage:
      "Yes — three beats of beamed eighths and then a quarter note.",
    usePitch: true,
    playKind: "per-beat"
  },
  {
    sequence: [
      "quarter-note",
      "quarter-rest",
      "eighth-beam-pair",
      "quarter-note"
    ],
    bankClasses: ["quarter-note", "quarter-rest", "eighth-beam-pair"],
    slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
    successMessage:
      "Yes — quarter note, quarter rest, beamed eighths, quarter note.",
    usePitch: true,
    playKind: "per-beat"
  }
]
},
"sixteenth-beam-pair": {
  exercises: [
    {
      sequence: [
        "sixteenth-beam-pair",
        "compound-eighth-rest-two-sixteenths",
        "compound-eighth-rest-note",
        "compound-two-sixteenths-eighth-rest"
      ],
      bankClasses: [
        "quarter-note",
        "quarter-rest",

```



```

      "eighth-beam-pair",
      "compound-eighth-note-rest",
      "compound-eighth-rest-note",
      "sixteenth-beam-pair",
      "compound-eighth-rest-two-sixteenths",
      "compound-two-sixteenths-eighth-rest"
    ],
    slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
    successMessage:
      "Yes — beamed sixteenths, eighth rest + two sixteenths, eighth rest + eighth note, two
sixteenths + eighth rest.",
    usePitch: true,
    playKind: "per-beat"
  },
  {
    sequence: [
      "sixteenth-beam-pair",
      "sixteenth-beam-pair",
      "eighth-beam-pair",
      "compound-two-sixteenths-eighth-rest"
    ],
    bankClasses: [
      "quarter-note",
      "quarter-rest",
      "eighth-beam-pair",
      "compound-eighth-note-rest",
      "compound-eighth-rest-note",
      "sixteenth-beam-pair",
      "compound-eighth-rest-two-sixteenths",
      "compound-two-sixteenths-eighth-rest"
    ],
    slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
    successMessage:
      "Yes — two beats of beamed sixteenths, beamed eighths, then two sixteenths + eighth
rest.",
    usePitch: true,
    playKind: "per-beat"
  }
],
"sixteenth-note": {
  exercises: (function () {
    const pool = [
      "quarter-note",

```

```

    "quarter-rest",
    "eighth-beam-pair",
    "compound-eighth-note-rest",
    "compound-eighth-rest-note",
    "sixteenth-beam-pair",
    "compound-eighth-rest-two-sixteenths",
    "compound-two-sixteenths-eighth-rest"
  ];
  const makeDrill = function () {
    return {
      randomFromPool: true,
      sequenceLength: 4,
      pool: pool.slice(),
      bankClasses: pool.slice(),
      slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
      successMessage: "Correct!",
      usePitch: true,
      playKind: "per-beat"
    };
  };
  return [makeDrill(), makeDrill(), makeDrill()];
})();
},
"sixteenth-rest": {
  exercises: (function () {
    const pool = [
      "quarter-note",
      "quarter-rest",
      "eighth-beam-pair",
      "compound-eighth-note-rest",
      "compound-eighth-rest-note",
      "sixteenth-beam-pair",
      "compound-eighth-rest-two-sixteenths",
      "compound-two-sixteenths-eighth-rest",
      "compound-sixteenth-rest-three-sixteenths"
    ];
    const makeDrill = function () {
      return {
        randomFromPool: true,
        sequenceLength: 4,
        pool: pool.slice(),
        bankClasses: pool.slice(),
        slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
        successMessage: "Correct!",

```

```

        usePitch: true,
        playKind: "per-beat"
    };
};
return [makeDrill(), makeDrill(), makeDrill()];
})();
},
"rhythm-eighth-two-sixteenths": (function () {
    const pool = [
        "quarter-note",
        "quarter-rest",
        "eighth-beam-pair",
        "compound-eighth-note-rest",
        "compound-eighth-rest-note",
        "sixteenth-beam-pair",
        "compound-eighth-rest-two-sixteenths",
        "compound-two-sixteenths-eighth-rest",
        "compound-sixteenth-rest-three-sixteenths",
        "rhythm-eighth-two-sixteenths"
    ];
    const makeDrill = function () {
        return {
            bankClasses: pool.slice(),
            slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
            successMessage: "Correct!",
            usePitch: true,
            playKind: "per-beat"
        };
    };
    return {
        groupRandomConfig: {
            pool: pool.slice(),
            sequenceLength: 4,
            requireAtLeastOne: ["rhythm-eighth-two-sixteenths"]
        },
        exercises: [makeDrill(), makeDrill(), makeDrill()]
    };
})();
"rhythm-two-sixteenths-eighth": (function () {
    const pool = [
        "quarter-note",
        "quarter-rest",
        "eighth-beam-pair",
        "compound-eighth-note-rest",

```

```

    "compound-eighth-rest-note",
    "sixteenth-beam-pair",
    "compound-eighth-rest-two-sixteenths",
    "compound-two-sixteenths-eighth-rest",
    "compound-sixteenth-rest-three-sixteenths",
    "rhythm-eighth-two-sixteenths",
    "rhythm-two-sixteenths-eighth"
  ];
  const makeDrill = function () {
    return {
      bankClasses: pool.slice(),
      slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
      successMessage: "Correct!",
      usePitch: true,
      playKind: "per-beat"
    };
  };
  return {
    groupRandomConfig: {
      pool: pool.slice(),
      sequenceLength: 4,
      requireAtLeastOne: ["rhythm-two-sixteenths-eighth"]
    },
    exercises: [makeDrill(), makeDrill(), makeDrill()]
  };
})();
"rhythm-dotted-eighth-sixteenth": (function () {
  const pool = [
    "quarter-note",
    "quarter-rest",
    "eighth-beam-pair",
    "compound-eighth-note-rest",
    "compound-eighth-rest-note",
    "sixteenth-beam-pair",
    "compound-eighth-rest-two-sixteenths",
    "compound-two-sixteenths-eighth-rest",
    "compound-sixteenth-rest-three-sixteenths",
    "rhythm-eighth-two-sixteenths",
    "rhythm-two-sixteenths-eighth",
    "rhythm-dotted-eighth-sixteenth"
  ];
  const makeDrill = function () {
    return {
      bankClasses: pool.slice(),

```

```

    slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
    successMessage: "Correct!",
    usePitch: true,
    playKind: "per-beat"
  };
};
return {
  groupRandomConfig: {
    pool: pool.slice(),
    sequenceLength: 4,
    requireAtLeastOne: ["rhythm-dotted-eighth-sixteenth"]
  },
  exercises: [makeDrill(), makeDrill(), makeDrill()]
};
})();
"rhythm-sixteenth-dotted-eighth": (function () {
  const pool = [
    "quarter-note",
    "quarter-rest",
    "eighth-beam-pair",
    "compound-eighth-note-rest",
    "compound-eighth-rest-note",
    "sixteenth-beam-pair",
    "compound-eighth-rest-two-sixteenths",
    "compound-two-sixteenths-eighth-rest",
    "compound-sixteenth-rest-three-sixteenths",
    "rhythm-eighth-two-sixteenths",
    "rhythm-two-sixteenths-eighth",
    "rhythm-dotted-eighth-sixteenth",
    "rhythm-sixteenth-dotted-eighth"
  ];
  const makeDrill = function () {
    return {
      bankClasses: pool.slice(),
      slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
      successMessage: "Correct!",
      usePitch: true,
      playKind: "per-beat"
    };
  };
  return {
    groupRandomConfig: {
      pool: pool.slice(),
      sequenceLength: 4,

```

```

    requireAtLeastOne: ["rhythm-sixteenth-dotted-eighth"]
  },
  exercises: [makeDrill(), makeDrill(), makeDrill()]
};
})();
"rhythm-syncopation": (function () {
  const pool = [
    "quarter-note",
    "quarter-rest",
    "eighth-beam-pair",
    "compound-eighth-note-rest",
    "compound-eighth-rest-note",
    "sixteenth-beam-pair",
    "compound-eighth-rest-two-sixteenths",
    "compound-two-sixteenths-eighth-rest",
    "compound-sixteenth-rest-three-sixteenths",
    "rhythm-eighth-two-sixteenths",
    "rhythm-two-sixteenths-eighth",
    "rhythm-dotted-eighth-sixteenth",
    "rhythm-sixteenth-dotted-eighth",
    "rhythm-syncopation"
  ];
  const makeDrill = function () {
    return {
      bankClasses: pool.slice(),
      slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
      successMessage: "Correct!",
      usePitch: true,
      playKind: "per-beat"
    };
  };
  return {
    groupRandomConfig: {
      pool: pool.slice(),
      sequenceLength: 4,
      requireAtLeastOne: ["rhythm-syncopation"]
    },
    exercises: [makeDrill(), makeDrill(), makeDrill()]
  };
})();
"rhythm-trainer": {
  infinite: true,
  exercises: [
    {

```

```

    randomFromWeightedSets: true,
    drawBeat: drawRhythmTrainerBeat,
    getBankClasses: rhythmTrainerBankClasses,
    sequenceLength: 4,
    slotLabels: ["Beat 1", "Beat 2", "Beat 3", "Beat 4"],
    successMessage: "Correct!",
    usePitch: true,
    playKind: "per-beat"
  }
]
}
};

```

```

let exerciseSourceClass = "quarter-note";
let exerciseStepIndex = 0;
let exerciseGroupTotalChecks = 0;
let exerciseGroupCorrectChecks = 0;

```

```

let exerciseSequenceToken = 0;
let draggedExerciseChip = null;
/** After the first wrong Check on this exercise step, "Play exercises" stays off until Next. */
let exerciseReplayDisallowed = false;
/**

```

```

 * Resolved sequence for the current exercise step. For static specs this is a
 * copy of `spec.sequence`; for `randomFromPool` specs it's freshly drawn from
 * `spec.pool` when the step is entered; for series with `groupRandomConfig`
 * it's pre-generated for the whole group and indexed by `exerciseStepIndex`.
 */

```

```

let exerciseRuntimeSequence = null;
/**

```

```

 * Pre-generated random sequences (one per exercise step) for series whose
 * entry sets `groupRandomConfig`. Cleared when the panel re-initializes.
 */

```

```

let exerciseGroupSequences = null;

```

```

function currentExerciseSequence() {
  return exerciseRuntimeSequence;
}

```

```

function resolveExerciseSequence(spec) {
  if (spec.randomFromWeightedSets && typeof spec.drawBeat === "function") {
    const len = spec.sequenceLength || (spec.slotLabels ? spec.slotLabels.length : 4);
    const seq = [];
    for (let k = 0; k < len; k += 1) {

```

```

    const beat = spec.drawBeat();
    if (beat) {
        seq.push(beat);
    }
}
return seq;
}
if (spec.randomFromPool && spec.pool && spec.pool.length) {
    const len = spec.sequenceLength || (spec.slotLabels ? spec.slotLabels.length : 4);
    const seq = [];
    for (let k = 0; k < len; k += 1) {
        const idx = Math.floor(Math.random() * spec.pool.length);
        seq.push(spec.pool[idx]);
    }
    return seq;
}
return spec.sequence ? spec.sequence.slice() : [];
}

```

```

function generateGroupSequences(groupCfg, exerciseCount) {
    const length = groupCfg.sequenceLength || 4;
    const pool = groupCfg.pool || [];
    if (!pool.length || exerciseCount <= 0) {
        return [];
    }
    const total = exerciseCount * length;
    const allBeats = [];
    for (let i = 0; i < total; i += 1) {
        const idx = Math.floor(Math.random() * pool.length);
        allBeats.push(pool[idx]);
    }
    if (groupCfg.requireAtLeastOne) {
        groupCfg.requireAtLeastOne.forEach(function (cls) {
            if (allBeats.indexOf(cls) === -1) {
                const pos = Math.floor(Math.random() * total);
                allBeats[pos] = cls;
            }
        });
    }
    const sequences = [];
    for (let e = 0; e < exerciseCount; e += 1) {
        sequences.push(allBeats.slice(e * length, (e + 1) * length));
    }
    return sequences;
}

```



```
}
```

```
function releaseExercisePlayButtonAfterPlayback() {  
  exercisePlayBtn.disabled = exerciseReplayDisallowed;  
}
```

```
const BPM_MIN = 40;  
const BPM_MAX = 240;  
const BPM_STEP = 5;  
let bpm = 80;  
let playbackEndTimerId = null;  
let playbackGen = 0;  
let rhythmFlashTimeoutIds = [];  
let isMetronomeRunning = false;  
let audioContext = null;  
let practiceOutGain = null;  
let activeRhythmClass = "";
```

```
function clampBpm(n) {  
  return Math.min(BPM_MAX, Math.max(BPM_MIN, n));  
}
```

```
function renderBpm() {  
  bpmValueEl.textContent = String(bpm);  
}  
renderBpm();
```

```
function ensureAudioContext() {  
  if (!audioContext) {  
    audioContext = new (window.AudioContext || window.webkitAudioContext)();  
  }  
  if (audioContext.state === "suspended") {  
    audioContext.resume();  
  }  
}
```

```
function ensurePracticeOutGain() {  
  ensureAudioContext();  
  if (!practiceOutGain) {  
    practiceOutGain = audioContext.createGain();  
    practiceOutGain.gain.value = 1;  
    practiceOutGain.connect(audioContext.destination);  
  }  
}
```

```

function playMetronomeClick() {
  if (!audioContext) {
    return;
  }
  const now = audioContext.currentTime;
  const osc = audioContext.createOscillator();
  const gain = audioContext.createGain();
  osc.type = "sine";
  osc.frequency.setValueAtTime(1400, now);
  gain.gain.setValueAtTime(0.0001, now);
  gain.gain.exponentialRampToValueAtTime(0.025, now + 0.005);
  gain.gain.exponentialRampToValueAtTime(0.0001, now + 0.06);
  osc.connect(gain);
  gain.connect(audioContext.destination);
  osc.start(now);
  osc.stop(now + 0.07);
}

function flashElement(el) {
  el.classList.add("flash");
  window.setTimeout(function () {
    el.classList.remove("flash");
  }, 90);
}

function silencePracticeOutput() {
  if (!audioContext || !practiceOutGain) {
    return;
  }
  const t = audioContext.currentTime;
  practiceOutGain.gain.cancelScheduledValues(t);
  practiceOutGain.gain.setValueAtTime(practiceOutGain.gain.value, t);
  practiceOutGain.gain.linearRampToValueAtTime(0.0001, t + 0.06);
}

function armPracticeOutput() {
  if (!audioContext || !practiceOutGain) {
    return;
  }
  const t = audioContext.currentTime;
  practiceOutGain.gain.cancelScheduledValues(t);
  practiceOutGain.gain.setValueAtTime(1, t);
}

```

```

function stopPlayback() {
  playbackGen += 1;
  rhythmFlashTimeoutIds.forEach(function (id) {
    window.clearTimeout(id);
  });
  rhythmFlashTimeoutIds = [];
  if (playbackEndTimerId !== null) {
    window.clearTimeout(playbackEndTimerId);
    playbackEndTimerId = null;
  }
  isMetronomeRunning = false;
  bpmFlashBar.classList.remove("flash");
  rhythmFlashBar.classList.remove("flash");
  rhythmFlashBar.classList.remove("sustain");
  silencePracticeOutput();
}

```

```

const RHYTHM_PITCH_SEGMENTS = {
  "eighth-beam-pair": [
    { start: 0, dur: 2 },
    { start: 2, dur: 2 }
  ],
  "sixteenth-beam-pair": [
    { start: 0, dur: 1 },
    { start: 1, dur: 1 },
    { start: 2, dur: 1 },
    { start: 3, dur: 1 }
  ],
  "rhythm-eighth-two-sixteenths": [
    { start: 0, dur: 2 },
    { start: 2, dur: 1 },
    { start: 3, dur: 1 }
  ],
  "rhythm-two-sixteenths-eighth": [
    { start: 0, dur: 1 },
    { start: 1, dur: 1 },
    { start: 2, dur: 2 }
  ],
  "rhythm-dotted-eighth-sixteenth": [
    { start: 0, dur: 3 },
    { start: 3, dur: 1 }
  ],
  "rhythm-sixteenth-dotted-eighth": [

```

```

    { start: 0, dur: 1 },
    { start: 1, dur: 3 }
  ],
  "rhythm-syncopation": [
    { start: 0, dur: 1 },
    { start: 1, dur: 2 },
    { start: 3, dur: 1 }
  ],
  "compound-eighth-note-rest": [{ start: 0, dur: 2 }],
  "compound-eighth-rest-note": [{ start: 2, dur: 2 }],
  "compound-eighth-rest-two-sixteenths": [
    { start: 2, dur: 1 },
    { start: 3, dur: 1 }
  ],
  "compound-two-sixteenths-eighth-rest": [
    { start: 0, dur: 1 },
    { start: 1, dur: 1 }
  ],
  "compound-sixteenth-rest-three-sixteenths": [
    { start: 1, dur: 1 },
    { start: 2, dur: 1 },
    { start: 3, dur: 1 }
  ],
  "compound-sixteenth-rest-sixteenth-note-eighth-note": [
    { start: 1, dur: 1 },
    { start: 2, dur: 2 }
  ],
  "compound-sixteenth-rest-sixteenth-note-eighth-rest": [
    { start: 1, dur: 1 }
  ],
  "compound-eighth-rest-sixteenth-rest-sixteenth-note": [
    { start: 3, dur: 1 }
  ],
  "compound-sixteenth-note-eighth-rest-sixteenth-note": [
    { start: 0, dur: 1 },
    { start: 3, dur: 1 }
  ],
  "compound-sixteenth-rest-eighth-note-sixteenth-note": [
    { start: 1, dur: 2 },
    { start: 3, dur: 1 }
  ],
  "compound-sixteenth-rest-dotted-eighth": [
    { start: 1, dur: 3 }
  ],

```

```

"compound-sixteenth-note-sixteenth-rest-eighth-note": [
  { start: 0, dur: 1 },
  { start: 2, dur: 2 }
],
"compound-sixteenth-note-sixteenth-rest-eighth-rest": [
  { start: 0, dur: 1 }
],
"compound-eighth-rest-sixteenth-note-sixteenth-rest": [
  { start: 2, dur: 1 }
],
"compound-eighth-note-sixteenth-note-sixteenth-rest": [
  { start: 0, dur: 2 },
  { start: 2, dur: 1 }
],
"compound-eighth-note-sixteenth-rest-sixteenth-note": [
  { start: 0, dur: 2 },
  { start: 3, dur: 1 }
],
"compound-sixteenth-note-sixteenth-rest-sixteenth-note-sixteenth-rest": [
  { start: 0, dur: 1 },
  { start: 2, dur: 1 }
],
"compound-sixteenth-rest-two-sixteenths-sixteenth-rest": [
  { start: 1, dur: 1 },
  { start: 2, dur: 1 }
],
"compound-sixteenth-rest-sixteenth-note-sixteenth-rest-sixteenth-note": [
  { start: 1, dur: 1 },
  { start: 3, dur: 1 }
],
"compound-sixteenth-note-eighth-note-sixteenth-rest": [
  { start: 0, dur: 1 },
  { start: 1, dur: 2 }
],
"compound-dotted-eighth-sixteenth-rest": [
  { start: 0, dur: 3 }
],
"compound-sixteenth-rest-eighth-note-sixteenth-rest": [
  { start: 1, dur: 2 }
]
];

```

```

function rhythmPitchSegments(rhythmClass) {
  return RHYTHM_PITCH_SEGMENTS[rhythmClass] || null;
}

```

```
}
```

```
function lessonPitchHz() {  
  return Number(quarterFreqInput.value) || 440;  
}
```

```
function rhythmUsesLessonPitch(rhythmClass) {  
  return (  
    rhythmClass === "quarter-note" ||  
    rhythmClass === "eighth-note" ||  
    rhythmClass === "sixteenth-note" ||  
    rhythmClass === "dotted-eighth" ||  
    rhythmClass === "sixteenth-rest" ||  
    !!RHYTHM_PITCH_SEGMENTS[rhythmClass]  
  );  
}
```

```
const RHYTHM_GAIN_FLOOR = 0.0001;  
const RHYTHM_GAIN_PEAK = 0.055;
```

```
/**
```

```
 * Short attack, long sustain near peak, then linear release at the tail so  
 * solo notes (quarter, single eighth, etc.) occupy their full geometric  
 * duration audibly while still fading out gracefully at the end.  
 */
```

```
function rhythmSineGainEnvelope(gainNode, startT, durationSec) {  
  const floor = RHYTHM_GAIN_FLOOR;  
  const peak = RHYTHM_GAIN_PEAK;  
  const end = startT + durationSec;  
  if (durationSec <= 0.012) {  
    gainNode.gain.setValueAtTime(floor, startT);  
    gainNode.gain.linearRampToValueAtTime(peak, startT + durationSec * 0.45);  
    gainNode.gain.linearRampToValueAtTime(floor, end);  
    return;  
  }  
  const atk = Math.min(0.025, Math.max(0.003, durationSec * 0.12));  
  const releaseDur = Math.min(0.18, Math.max(0.02, durationSec * 0.3));  
  const atkEnd = startT + atk;  
  const sustainEnd = end - releaseDur;  
  if (sustainEnd <= atkEnd + 0.002) {  
    gainNode.gain.setValueAtTime(floor, startT);  
    gainNode.gain.linearRampToValueAtTime(peak, atkEnd);  
    gainNode.gain.linearRampToValueAtTime(floor, end);  
    return;  
  }
```

```

    }
    gainNode.gain.setValueAtTime(floor, startT);
    gainNode.gain.linearRampToValueAtTime(peak, atkEnd);
    gainNode.gain.linearRampToValueAtTime(peak, sustainEnd);
    gainNode.gain.linearRampToValueAtTime(floor, end);
}

/**
 * Tight envelope for segmented rhythms (beamed eighths/sixteenths and
 * compound chips). Each segment should fill almost all of its slot so the
 * beat doesn't sound like it stops between notes; the release is a brief
 * tail (~5% of the note's duration, capped at 15 ms) and the attack is also
 * short so adjacent segments butt up against each other cleanly.
 */
function rhythmSineGainEnvelopeTight(gainNode, startT, durationSec) {
    const floor = RHYTHM_GAIN_FLOOR;
    const peak = RHYTHM_GAIN_PEAK;
    const end = startT + durationSec;
    if (durationSec <= 0.012) {
        gainNode.gain.setValueAtTime(floor, startT);
        gainNode.gain.linearRampToValueAtTime(peak, startT + durationSec * 0.5);
        gainNode.gain.linearRampToValueAtTime(floor, end);
        return;
    }
    const atk = Math.min(0.015, Math.max(0.003, durationSec * 0.08));
    const releaseDur = Math.min(0.015, Math.max(0.006, durationSec * 0.05));
    const atkEnd = startT + atk;
    const sustainEnd = end - releaseDur;
    if (sustainEnd <= atkEnd + 0.002) {
        gainNode.gain.setValueAtTime(floor, startT);
        gainNode.gain.linearRampToValueAtTime(peak, atkEnd);
        gainNode.gain.linearRampToValueAtTime(floor, end);
        return;
    }
    gainNode.gain.setValueAtTime(floor, startT);
    gainNode.gain.linearRampToValueAtTime(peak, atkEnd);
    gainNode.gain.linearRampToValueAtTime(peak, sustainEnd);
    gainNode.gain.linearRampToValueAtTime(floor, end);
}

function schedulePitchedToneAt(startAudioTime, durationSec, freqHz) {
    if (!audioContext || durationSec <= 0.002) {
        return;
    }
}

```

```

ensurePracticeOutGain();
const osc = audioContext.createOscillator();
const gain = audioContext.createGain();
osc.type = "sine";
osc.frequency.setValueAtTime(freqHz, startAudioTime);
rhythmSineGainEnvelopeTight(gain, startAudioTime, durationSec);
osc.connect(gain);
gain.connect(practiceOutGain);
osc.start(startAudioTime);
osc.stop(startAudioTime + durationSec + 0.02);
}

```

```

function playPitchedRhythmSegments(segments, beatSec, freqHz) {
  if (!audioContext || !segments || !segments.length) {
    return;
  }
  ensurePracticeOutGain();
  armPracticeOutput();
  const now = audioContext.currentTime;
  const step = beatSec / 4;
  segments.forEach(function (seg) {
    const dur = seg.dur * step;
    const durMs = dur * 1000;
    schedulePitchedToneAt(now + seg.start * step, dur, freqHz);
    const delayMs = seg.start * step * 1000;
    if (delayMs <= 0) {
      triggerRhythmBarSustain(durMs);
    } else {
      const tid = window.setTimeout(function () {
        triggerRhythmBarSustain(durMs);
      }, delayMs);
      rhythmFlashTimeoutIds.push(tid);
    }
  });
}

```

```

function triggerRhythmBarSustain(durationMs) {
  const ms = durationMs != null ? durationMs : 60000 / bpm;
  rhythmFlashBar.style.setProperty("--beat-duration", ms + "ms");
  rhythmFlashBar.classList.remove("sustain");
  void rhythmFlashBar.offsetWidth;
  rhythmFlashBar.classList.add("sustain");
}

```



```

function playOneBeatForClass(rhythmClass, opts) {
  opts = opts || {};
  const freqHz = opts.freqHz != null ? opts.freqHz : 440;
  const beatFraction =
    opts.beatFraction != null ? opts.beatFraction : 1;
  const skipEndTimer = opts.skipEndTimer === true;
  if (!opts.skipStop) {
    stopPlayback();
  } else {
    if (playbackEndTimerId !== null) {
      window.clearTimeout(playbackEndTimerId);
      playbackEndTimerId = null;
    }
  }
  ensureAudioContext();
  ensurePracticeOutGain();
  isMetronomeRunning = true;
  const beatMs = (60000 / bpm) * beatFraction;
  const beatSec = beatMs / 1000;

  flashElement(bpmFlashBar);
  playMetronomeClick();

  if (rhythmClass === "quarter-note") {
    triggerRhythmBarSustain(beatMs);
    armPracticeOutput();
    playQuarterNoteTone(beatSec, freqHz);
  } else if (rhythmClass === "eighth-note") {
    triggerRhythmBarSustain(beatMs);
    armPracticeOutput();
    playQuarterNoteTone(beatSec, freqHz);
  } else if (rhythmClass === "sixteenth-note") {
    triggerRhythmBarSustain(beatMs);
    armPracticeOutput();
    playQuarterNoteTone(beatSec, freqHz);
  } else if (rhythmClass === "dotted-eighth") {
    triggerRhythmBarSustain(beatMs);
    armPracticeOutput();
    playQuarterNoteTone(beatSec, freqHz);
  } else if (rhythmClass === "sixteenth-rest") {
    armPracticeOutput();
  } else {
    const segments = rhythmPitchSegments(rhythmClass);
    if (segments && segments.length > 0) {

```

```

    playPitchedRhythmSegments(segments, beatSec, freqHz);
  }
}

if (!skipEndTimer) {
  playbackEndTimerId = window.setTimeout(function () {
    playbackEndTimerId = null;
    stopPlayback();
  }, beatMs);
}
}

function playOneBeat() {
  if (!activeRhythmClass) {
    return;
  }
  const freq = lessonPitchHz();
  const opts = { freqHz: freq };
  if (activeRhythmClass === "eighth-note") {
    opts.beatFraction = 0.5;
  } else if (activeRhythmClass === "sixteenth-note") {
    opts.beatFraction = 0.25;
  } else if (activeRhythmClass === "sixteenth-rest") {
    opts.beatFraction = 0.25;
  } else if (activeRhythmClass === "dotted-eighth") {
    opts.beatFraction = 0.75;
  }
  playOneBeatForClass(activeRhythmClass, opts);
}

function playQuarterNoteTone(durationSec, freqHz) {
  ensurePracticeOutGain();
  armPracticeOutput();
  const now = audioContext.currentTime;
  const osc = audioContext.createOscillator();
  const gain = audioContext.createGain();
  osc.type = "sine";
  osc.frequency.setValueAtTime(freqHz, now);
  rhythmSineGainEnvelope(gain, now, durationSec);
  osc.connect(gain);
  gain.connect(practiceOutGain);
  osc.start(now);
  osc.stop(now + durationSec + 0.02);
}

```

```

function adjustBpm(delta) {
  bpm = clampBpm(bpm + delta);
  renderBpm();
  if (playbackEndTimerId !== null) {
    stopPlayback();
  }
}

```

```

bpmMinus.addEventListener("click", function () {
  adjustBpm(-BPM_STEP);
});
bpmPlus.addEventListener("click", function () {
  adjustBpm(BPM_STEP);
});
const rhythmDetails = {
  "quarter-note": {
    first: "Well, you want to start at the beginning, huh?\nThis is a quarter note. It is the
simplest possible rhythm: one beat, nothing complicated.\nPress the play button to see and
hear what a quarter note is.",
    second: "Haha! Here is an exercise!"
  },
  "quarter-rest": {
    first: "All rests are \"counterparts\" of notes.\nA note and its corresponding rest form a pair
with the same duration, just that there is sound for the note, but not for the rest.\nPress the play
button to see what I mean.",
    second: "Next are exercises: you will drag quarter notes and quarter rests into the beats
you hear."
  },
  "eighth-beam-pair": {
    first: "Pretty boring, huh?\nHere's something to break up the monotonicity. The beamed
eighth notes take up the same amount of time as the quarter, but are played twice as fast.",
    second: "The metronome's speed is customizable."
  },
  "eighth-note": {
    first: "This is a single eighth note.\nThe duration is half a beat.",
    second: "Oh yay! There are no exercises for this one!"
  },
  "eighth-rest": {
    first: "This is the \"rest counterpart\" of the single eighth note. Its duration is half a beat.\nAn
eighth note and an eighth rest, one after another, is not a quarter note. For the quarter note, you
must hold the note for the entire beat, while you must release halfway through the beat for the
eighth note + eighth rest.",

```

second: "More exercises for you! Hahahaha\n\nIn one beat, you can have multiple notes. For example, in the first beat, it has a single eighth for the first part of the beat and an eighth rest for the second part."

},

"dotted-eighth": {

first: "When you dot something, the duration multiplies by 3/2.\n\nFor this dotted eighth, this means that its duration is $1/2 \times 3/2 = 3/4$ of a beat.",

second: "There are temporarily no practices for this one."

},

"sixteenth-beam-pair": {

first: "These sure look familiar, do they?\n\nEach of these sixteenth notes is 1/4 of a beat. Together, they share the 1 beat they are allotted.",

second: "And, to make sure that you understand the notes all correctly, you must get at least 80% of your practices correct!"

},

"sixteenth-note": {

first: "This probably also looks very familiar. This is a single sixteenth note.\n\nIts duration is 1/4 of a beat.",

second: "You can have two, three, or four sixteenth notes beamed together as well."

},

"sixteenth-rest": {

first: "This is the \u201Crest counterpart\u201D of the sixteenth note. The rest is self-explanatory."

},

"rhythm-eighth-two-sixteenths": {

first: "This is just an eighth followed by two sixteenth notes.\n\nIf you divide each beat into \u201Csubbeats\u201D that are each 1/4 of a beat, then you should play for the first, third, and fourth subbeats."

},

"rhythm-two-sixteenths-eighth": {

first: "This is similar to the previous one. Play for the first, second, and third subbeats."

},

"rhythm-dotted-eighth-sixteenth": {

first: "Self-explanatory. Play for the first and fourth subbeats."

},

"rhythm-sixteenth-dotted-eighth": {

first: "Play for the first and second subbeats."

},

"rhythm-syncopation": {

first: "Play for the first, second, and fourth subbeats.",

second: "After these is the rhythm trainer. It seems similar to a lot of exercises.\n\nWatch out for other rhythms, like eighth note + sixteenth rest + sixteenth note (I'll write that with the sixteenth rest under the beam), that I didn't go over.\n\nChoose Challenging Mode if you are ready for it, otherwise use Original Mode."

```

    }
  };
  let activeSecondText = "";

  function syncQuarterFreqLabel() {
    quarterFreqValueEl.textContent = String(Math.round(Number(quarterFreqInput.value) ||
440));
  }
  quarterFreqInput.addEventListener("input", syncQuarterFreqLabel);
  syncQuarterFreqLabel();

  function openRhythmLessonFromTile(tile) {
    detailPanel.classList.remove("know-all-detail");
    rhythmSession.classList.remove("know-all-detail");
    rhythmSession.classList.remove("exercises-mode");
    hideRhythmTrainerModeButtons();
    detailSymbol.innerHTML = "";
    const clone = tile.cloneNode(true);
    clone.setAttribute("width", "150");
    clone.setAttribute("height", "150");
    detailSymbol.appendChild(clone);
    const rhythmClass = tile.classList[0];
    activeRhythmClass = rhythmClass;
    const titleText = RHYTHM_DISPLAY_NAMES[rhythmClass] || "";
    detailTitle.textContent = titleText;
    detailTitle.hidden = !titleText;
    const selectedRhythm = rhythmDetails[rhythmClass] || { first: "" };
    detailText.textContent = selectedRhythm.first;
    activeSecondText = selectedRhythm.second || "";
    detailSecondText.hidden = true;
    detailSecondText.textContent = "";
    detailPlayButton.hidden = false;
    if (rhythmUsesLessonPitch(rhythmClass)) {
      quarterToneRow.classList.add("visible");
      quarterToneRow.setAttribute("aria-hidden", "false");
    } else {
      quarterToneRow.classList.remove("visible");
      quarterToneRow.setAttribute("aria-hidden", "true");
    }
    exercisesPanel.hidden = true;
    practicePanel.hidden = true;
    detailPanel.hidden = false;
    rhythmSession.hidden = false;
    renderBpm();
  }

```

```
stopPlayback();
syncDetailNextVisibility();
}
```

```
function syncDetailNextVisibility() {
  if (!activeRhythmClass) {
    detailNextBtn.hidden = true;
    return;
  }
  detailNextBtn.hidden = false;
  const entry = EXERCISE_CONFIG[activeRhythmClass];
  const hasExercises =
    entry && entry.exercises && entry.exercises.length > 0;
  detailNextBtn.setAttribute(
    "aria-label",
    hasExercises ? "Next: exercises" : "Next: next rhythm lesson"
  );
}
```

```
function advanceToNextRhythmLessonFromStrip(rhythmClassJustFinished) {
  exerciseGroupTotalChecks = 0;
  exerciseGroupCorrectChecks = 0;
  hideExerciseGroupFailScreen();
  if (rhythmClassJustFinished === "rhythm-syncopation") {
    showRhythmTrainerLanding();
    return;
  }
  exercisesPanel.hidden = true;
  rhythmSession.classList.remove("exercises-mode");
  const tiles = Array.from(
    document.querySelectorAll(".rhythm-scroll-inner svg")
  );
  let idx = -1;
  for (let i = 0; i < tiles.length; i += 1) {
    if (tiles[i].classList[0] === rhythmClassJustFinished) {
      idx = i;
      break;
    }
  }
  if (idx >= 0 && idx + 1 < tiles.length) {
    openRhythmLessonFromTile(tiles[idx + 1]);
  } else {
    rhythmSession.hidden = true;
    practicePanel.hidden = false;
  }
}
```

```
}  
}
```

```
function hideRhythmTrainerModeButtons() {  
    rhythmTrainerModeButtonsEl.hidden = true;  
}
```

```
/**
```

```
 * Show the rhythm-trainer landing page where the user picks Original or  
 * Challenging. Reached after passing syncopation's exercises and from the  
 * "I know all" button.
```

```
*/
```

```
function showRhythmTrainerLanding() {  
    stopPlayback();  
    exerciseSequenceToken += 1;  
    rhythmSession.hidden = false;  
    rhythmSession.classList.add("know-all-detail");  
    rhythmSession.classList.remove("exercises-mode");  
    detailPanel.classList.add("know-all-detail");  
    detailSymbol.innerHTML = "";  
    detailTitle.textContent = RHYTHM_DISPLAY_NAMES["rhythm-trainer"];  
    detailTitle.hidden = false;  
    activeRhythmClass = "rhythm-trainer";  
    detailText.textContent =  
        "Choose Challenging Mode if you are ready for it, otherwise use Original Mode.";  
    activeSecondText = "";  
    detailSecondText.hidden = true;  
    detailSecondText.textContent = "";  
    quarterToneRow.classList.remove("visible");  
    quarterToneRow.setAttribute("aria-hidden", "true");  
    practicePanel.hidden = true;  
    exercisesPanel.hidden = true;  
    detailPanel.hidden = false;  
    detailNextBtn.hidden = true;  
    rhythmTrainerModeButtonsEl.hidden = false;  
}
```

```
/**
```

```
 * Start the trainer's infinite exercises after the user picks a mode.
```

```
*/
```

```
function enterRhythmTrainerWithMode(mode) {  
    rhythmTrainerMode = RHYTHM_TRAINER_MODES[mode] ? mode : "original";  
    exerciseSequenceToken += 1;  
    stopPlayback();
```

```

exerciseSourceClass = "rhythm-trainer";
detailPanel.classList.remove("know-all-detail");
rhythmSession.classList.remove("know-all-detail");
detailPanel.hidden = true;
practicePanel.hidden = true;
rhythmSession.hidden = false;
exercisesPanel.hidden = false;
hideRhythmTrainerModeButtons();
initExercisesPanel();
}

function currentExerciseSpec() {
  const entry = EXERCISE_CONFIG[exerciseSourceClass];
  if (!entry || !entry.exercises || !entry.exercises.length) {
    return null;
  }
  return entry.exercises[exerciseStepIndex] || null;
}

function setupExerciseStep() {
  const c = currentExerciseSpec();
  if (!c) {
    return;
  }
  exerciseReplayDisallowed = false;
  releaseExercisePlayButtonAfterPlayback();
  if (
    exerciseGroupSequences &&
    exerciseGroupSequences[exerciseStepIndex]
  ) {
    exerciseRuntimeSequence = exerciseGroupSequences[exerciseStepIndex].slice();
  } else {
    exerciseRuntimeSequence = resolveExerciseSequence(c);
  }
  exerciseAnswerArea.hidden = false;
  exerciseFeedback.textContent = "";
  exerciseNextBtn.hidden = true;
  renderExerciseSlots(c.slotLabels);
  if (c.usePitch) {
    quarterToneRow.classList.add("visible");
    quarterToneRow.setAttribute("aria-hidden", "false");
  } else {
    quarterToneRow.classList.remove("visible");
    quarterToneRow.setAttribute("aria-hidden", "true");
  }
}

```



```

    }
    if (c.bankLabel) {
        exerciseBankLabelEl.textContent = c.bankLabel;
        exerciseBankLabelEl.hidden = false;
    } else {
        exerciseBankLabelEl.textContent = "";
        exerciseBankLabelEl.hidden = true;
    }
    const bankClasses =
        typeof c.getBankClasses === "function"
            ? c.getBankClasses()
            : c.bankClasses;
    initExerciseDragBank(bankClasses || []);
    exerciseNextBtn.textContent = "Next";
    exerciseNextBtn.setAttribute("aria-label", "Continue");
}

function hideExerciseGroupFailScreen() {
    exerciseGroupFailEl.hidden = true;
    exerciseMainBlockEl.hidden = false;
}

function showExerciseGroupFailScreen() {
    exerciseGroupFailEl.hidden = false;
    exerciseMainBlockEl.hidden = true;
}

function refreshSlotFilledClass() {
    exerciseSlotsEl.querySelectorAll(".drop-zone").forEach(function (z) {
        z.classList.toggle("filled", z.querySelector(".rhythm-drag") !== null);
    });
}

function clearExerciseSlotContents() {
    exerciseSlotsEl.querySelectorAll(".drop-zone").forEach(function (z) {
        z.innerHTML = "";
        z.classList.remove("drop-hover", "filled");
    });
    refreshSlotFilledClass();
}

function renderExerciseSlots(slotLabels) {
    exerciseSlotsEl.innerHTML = "";
    exerciseSlotsEl.classList.toggle("slots-two", slotLabels.length === 2);
}

```

```

exerciseSlotsEl.setAttribute(
  "aria-label",
  "Answer boxes: " + slotLabels.join(", ")
);
slotLabels.forEach(function (label, i) {
  const slot = document.createElement("div");
  slot.className = "exercise-slot";
  const lab = document.createElement("span");
  lab.className = "slot-label";
  lab.textContent = label;
  const zone = document.createElement("div");
  zone.className = "drop-zone";
  zone.setAttribute("data-slot", String(i));
  slot.appendChild(lab);
  slot.appendChild(zone);
  exerciseSlotsEl.appendChild(slot);
});
}

function scaleExerciseSvgClone(src, targetW) {
  const copy = src.cloneNode(true);
  const sw = parseFloat(src.getAttribute("width")) || "42";
  const sh = parseFloat(src.getAttribute("height")) || "51";
  const tw = targetW != null ? targetW : 48;
  copy.setAttribute("width", String(tw));
  copy.setAttribute("height", String(Math.round(sh * (tw / sw))));
  return copy;
}

const COMPOUND_CHIP_GLYPHS = {
  "compound-eighth-note-rest": ["eighth-note", "eighth-rest"],
  "compound-eighth-rest-note": ["eighth-rest", "eighth-note"],
  "compound-eighth-rest-two-sixteenths": [
    "eighth-rest",
    "two-sixteenths-beamed"
  ],
  "compound-two-sixteenths-eighth-rest": [
    "two-sixteenths-beamed",
    "eighth-rest"
  ],
  "compound-sixteenth-rest-three-sixteenths": [
    "sixteenth-rest",
    "three-sixteenths-beamed"
  ],

```

```
"compound-sixteenth-rest-sixteenth-note-eighth-note": [  
  "sixteenth-rest",  
  "sixteenth-note",  
  "eighth-note"  
],  
"compound-sixteenth-rest-sixteenth-note-eighth-rest": [  
  "sixteenth-rest",  
  "sixteenth-note",  
  "eighth-rest"  
],  
"compound-eighth-rest-sixteenth-rest-sixteenth-note": [  
  "eighth-rest",  
  "sixteenth-rest",  
  "sixteenth-note"  
],  
"compound-sixteenth-note-eighth-rest-sixteenth-note": [  
  "sixteenth-note",  
  "eighth-rest",  
  "sixteenth-note"  
],  
"compound-sixteenth-rest-eighth-note-sixteenth-note": [  
  "sixteenth-rest",  
  "eighth-note",  
  "sixteenth-note"  
],  
"compound-sixteenth-rest-dotted-eighth": [  
  "sixteenth-rest",  
  "dotted-eighth"  
],  
"compound-sixteenth-note-sixteenth-rest-eighth-note": [  
  "sixteenth-note",  
  "sixteenth-rest",  
  "eighth-note"  
],  
"compound-sixteenth-note-sixteenth-rest-eighth-rest": [  
  "sixteenth-note",  
  "sixteenth-rest",  
  "eighth-rest"  
],  
"compound-eighth-rest-sixteenth-note-sixteenth-rest": [  
  "eighth-rest",  
  "sixteenth-note",  
  "sixteenth-rest"  
],
```

```

"compound-eighth-note-sixteenth-note-sixteenth-rest": [
    "eighth-note",
    "sixteenth-note",
    "sixteenth-rest"
],
"compound-eighth-note-sixteenth-rest-sixteenth-note": [
    "eighth-note",
    "sixteenth-rest",
    "sixteenth-note"
],
"compound-sixteenth-note-sixteenth-rest-sixteenth-note-sixteenth-rest": [
    "sixteenth-note",
    "sixteenth-rest",
    "sixteenth-note",
    "sixteenth-rest"
],
"compound-sixteenth-rest-two-sixteenths-sixteenth-rest": [
    "sixteenth-rest",
    "two-sixteenths-beamed",
    "sixteenth-rest"
],
"compound-sixteenth-rest-sixteenth-note-sixteenth-rest-sixteenth-note": [
    "sixteenth-rest",
    "sixteenth-note",
    "sixteenth-rest",
    "sixteenth-note"
],
"compound-sixteenth-note-eighth-note-sixteenth-rest": [
    "sixteenth-note",
    "eighth-note",
    "sixteenth-rest"
],
"compound-dotted-eighth-sixteenth-rest": [
    "dotted-eighth",
    "sixteenth-rest"
],
"compound-sixteenth-rest-eighth-note-sixteenth-rest": [
    "sixteenth-rest",
    "eighth-note",
    "sixteenth-rest"
]
];

/**

```

- * Chips rendered as a single beamed group (notes share a beam) with rests
- * placed appropriately:
 - * - Rests between the leftmost and rightmost note of the beam group sit UNDER the beam (smaller glyph).
 - * - Rests before/after the beam group are drawn standalone (full-size glyph, at the position's standard staff y).
- * Mixed eighth/sixteenth beam groups get a short half-beam attached to each sixteenth's stem pointing toward the eighth (a la syncopation).
- * Each item is `{ kind: "note"|"rest", noteDur|restDur: 1|2, pos: 0..3 }`.

```

const BEAMED_REST_CHIP_LAYOUTS = {
  "compound-sixteenth-note-eighth-rest-sixteenth-note": [
    { kind: "note", noteDur: 1, pos: 0 },
    { kind: "rest", restDur: 2, pos: 1 },
    { kind: "note", noteDur: 1, pos: 3 }
  ],
  "compound-sixteenth-note-sixteenth-rest-eighth-note": [
    { kind: "note", noteDur: 1, pos: 0 },
    { kind: "rest", restDur: 1, pos: 1 },
    { kind: "note", noteDur: 2, pos: 2 }
  ],
  "compound-eighth-note-sixteenth-rest-sixteenth-note": [
    { kind: "note", noteDur: 2, pos: 0 },
    { kind: "rest", restDur: 1, pos: 2 },
    { kind: "note", noteDur: 1, pos: 3 }
  ],
  "compound-sixteenth-rest-sixteenth-note-eighth-note": [
    { kind: "rest", restDur: 1, pos: 0 },
    { kind: "note", noteDur: 1, pos: 1 },
    { kind: "note", noteDur: 2, pos: 2 }
  ],
  "compound-sixteenth-rest-eighth-note-sixteenth-note": [
    { kind: "rest", restDur: 1, pos: 0 },
    { kind: "note", noteDur: 2, pos: 1 },
    { kind: "note", noteDur: 1, pos: 3 }
  ],
  "compound-eighth-note-sixteenth-note-sixteenth-rest": [
    { kind: "note", noteDur: 2, pos: 0 },
    { kind: "note", noteDur: 1, pos: 2 },
    { kind: "rest", restDur: 1, pos: 3 }
  ],
  "compound-sixteenth-note-eighth-note-sixteenth-rest": [
    { kind: "note", noteDur: 1, pos: 0 },
    { kind: "note", noteDur: 2, pos: 1 },

```

```

    { kind: "rest", restDur: 1, pos: 3 }
  ]
};

```

```

function buildBeamedRestGroupSvg(items, targetW) {
  const svgNS = "http://www.w3.org/2000/svg";
  const headCenters = [20, 54, 88, 122];
  const stemX = [35, 69, 103, 137];
  const baseW = 156;
  const baseH = 68;
  const tw = targetW != null ? targetW : 80;
  const th = Math.round(baseH * (tw / baseW));
  const svg = document.createElementNS(svgNS, "svg");
  svg.setAttribute("width", String(tw));
  svg.setAttribute("height", String(th));
  svg.setAttribute("viewBox", "0 -14 " + baseW + " " + baseH);
  svg.setAttribute("overflow", "visible");
  svg.setAttribute("aria-hidden", "true");
  function el(name, attrs) {
    const e = document.createElementNS(svgNS, name);
    Object.keys(attrs).forEach(function (k) {
      e.setAttribute(k, attrs[k]);
    });
    return e;
  }
  const noteItems = items
    .filter(function (it) {
      return it.kind === "note";
    })
    .sort(function (a, b) {
      return a.pos - b.pos;
    });
  noteItems.forEach(function (n) {
    const hx = headCenters[n.pos];
    const sx = stemX[n.pos];
    svg.appendChild(
      el("ellipse", {
        cx: String(hx),
        cy: "38",
        rx: "16",
        ry: "9",
        fill: "currentColor",
        transform: "rotate(-16 " + hx + " 38)"
      })
    )
  })
}

```

```

);
svg.appendChild(
  el("line", {
    x1: String(sx),
    y1: "-12",
    x2: String(sx),
    y2: "35",
    stroke: "currentColor",
    "stroke-width": "2",
    "stroke-linecap": "butt"
  })
);
});
if (noteItems.length >= 2) {
  const firstSx = stemX[noteItems[0].pos];
  const lastSx = stemX[noteItems[noteItems.length - 1].pos];
  const beamLen = lastSx - firstSx + 2;
  svg.appendChild(
    el("rect", {
      x: String(firstSx - 1),
      y: "-12",
      width: String(beamLen),
      height: "3",
      fill: "currentColor"
    })
  );
  const allSixteenths = noteItems.every(function (n) {
    return n.noteDur === 1;
  });
  if (allSixteenths) {
    svg.appendChild(
      el("rect", {
        x: String(firstSx - 1),
        y: "-7",
        width: String(beamLen),
        height: "3",
        fill: "currentColor"
      })
    );
  } else {
    const partialLen = 17;
    noteItems.forEach(function (n, i) {
      if (n.noteDur !== 1) {
        return;
      }
    });
  }
}

```

```

    }
    const sx = stemX[n.pos];
    let pStart, pWidth;
    if (i === 0 && notelItems[i + 1]) {
        pStart = sx - 1;
        pWidth = partialLen;
    } else if (notelItems[i - 1]) {
        pStart = sx + 1 - partialLen;
        pWidth = partialLen;
    } else {
        return;
    }
    svg.appendChild(
        el("rect", {
            x: String(pStart),
            y: "-7",
            width: String(pWidth),
            height: "3",
            fill: "currentColor"
        })
    );
    });
}
}
const minNotePos = notelItems.length ? notelItems[0].pos : -1;
const maxNotePos = notelItems.length
    ? notelItems[notelItems.length - 1].pos
    : -1;
items.forEach(function (it) {
    if (it.kind !== "rest") {
        return;
    }
    const restStart = it.pos;
    const restEnd = it.pos + it.restDur - 1;
    const inBeam =
        notelItems.length >= 2 &&
        restStart > minNotePos &&
        restEnd < maxNotePos;
    const xCenter = headCenters[it.pos] + 17 * (it.restDur - 1);
    const isEighth = it.restDur === 2;
    const restChar = isEighth ? "\ue4e6" : "\ue4e7";
    const restEl = el("text", {
        x: String(xCenter),
        y: inBeam ? "12" : "22",

```



```

    "font-size": inBeam ? "44" : "60",
    "text-anchor": "middle",
    "dominant-baseline": "central",
    "font-family": "Bravura, serif",
    fill: "currentColor"
  });
  restEl.textContent = restChar;
  svg.appendChild(restEl);
});
return svg;
}

```

```

function buildBeamedSixteenthsSvg(count, targetW) {
  const svgNS = "http://www.w3.org/2000/svg";
  const headCenters = [20, 54, 88, 122];
  const stemX = [35, 69, 103, 137];
  const n = Math.max(2, Math.min(4, count || 2));
  const lastStem = stemX[n - 1];
  const baseW = lastStem + 19;
  const baseH = 68;
  const tw =
    targetW != null ? targetW : Math.round(28 + n * 14);
  const th = Math.round(baseH * (tw / baseW));
  const svg = document.createElementNS(svgNS, "svg");
  svg.setAttribute("width", String(tw));
  svg.setAttribute("height", String(th));
  svg.setAttribute("viewBox", "0 -14 " + baseW + " " + baseH);
  svg.setAttribute("aria-hidden", "true");
  function el(name, attrs) {
    const e = document.createElementNS(svgNS, name);
    Object.keys(attrs).forEach(function (k) {
      e.setAttribute(k, attrs[k]);
    });
    return e;
  }
  for (let i = 0; i < n; i += 1) {
    svg.appendChild(
      el("ellipse", {
        cx: String(headCenters[i]),
        cy: "38",
        rx: "16",
        ry: "9",
        fill: "currentColor",
        transform: "rotate(-16 " + headCenters[i] + " 38)"
      })
    );
  }
}

```

```

    })
  );
}
const beamLen = stemX[n - 1] - stemX[0] + 2;
svg.appendChild(
  el("rect", {
    x: String(stemX[0] - 1),
    y: "-12",
    width: String(beamLen),
    height: "3",
    fill: "currentColor"
  })
);
svg.appendChild(
  el("rect", {
    x: String(stemX[0] - 1),
    y: "-7",
    width: String(beamLen),
    height: "3",
    fill: "currentColor"
  })
);
for (let i = 0; i < n; i += 1) {
  svg.appendChild(
    el("line", {
      x1: String(stemX[i]),
      y1: "-12",
      x2: String(stemX[i]),
      y2: "35",
      stroke: "currentColor",
      "stroke-width": "2",
      "stroke-linecap": "butt"
    })
  );
}
return svg;
}

function buildCompoundChip(rhythmClass) {
  const beamedLayout = BEAMED_REST_CHIP_LAYOUTS[rhythmClass];
  if (beamedLayout) {
    const beamedWrap = document.createElement("div");
    beamedWrap.className = "rhythm-drag";
    beamedWrap.setAttribute("data-rhythm-class", rhythmClass);
  }
}

```

```

    beamedWrap.setAttribute("draggable", "true");
    beamedWrap.appendChild(buildBeamedRestGroupSvg(beamedLayout, 76));
    return beamedWrap;
}
const glyphs = COMPOUND_CHIP_GLYPHS[rhythmClass];
if (!glyphs) {
    return null;
}
const wrap = document.createElement("div");
wrap.className = "rhythm-drag compound-chip";
wrap.setAttribute("data-rhythm-class", rhythmClass);
wrap.setAttribute("draggable", "true");
const baseW = glyphs.length === 2 ? 32 : 22;
for (let g = 0; g < glyphs.length; g += 1) {
    const glyphKey = glyphs[g];
    if (glyphKey === "two-sixteenths-beamed") {
        wrap.appendChild(buildBeamedSixteenthsSvg(2, 44));
        continue;
    }
    if (glyphKey === "three-sixteenths-beamed") {
        wrap.appendChild(buildBeamedSixteenthsSvg(3, 56));
        continue;
    }
    const src = document.querySelector(
        ".rhythm-scroll-inner svg." + glyphKey
    );
    if (!src) {
        return null;
    }
    wrap.appendChild(scaleExerciseSvgClone(src, baseW));
}
return wrap;
}

```

```

function initExerciseDragBank(bankClasses) {
    exerciseBank.innerHTML = "";
    exerciseFeedback.textContent = "";
    bankClasses.forEach(function (rhythmClass) {
        if (
            COMPOUND_CHIP_GLYPHS[rhythmClass] ||
            BEAMED_REST_CHIP_LAYOUTS[rhythmClass]
        ) {
            const compoundWrap = buildCompoundChip(rhythmClass);
            if (compoundWrap) {

```

```

        exerciseBank.appendChild(compoundWrap);
    }
    return;
}
const src = document.querySelector(
    ".rhythm-scroll-inner svg." + rhythmClass
);
if (!src) {
    return;
}
const wide =
    rhythmClass.indexOf("rhythm-") === 0 ||
    rhythmClass === "sixteenth-beam-pair" ||
    rhythmClass === "eighth-beam-pair";
const targetW = wide ? 56 : 48;
const wrap = document.createElement("div");
wrap.className = "rhythm-drag";
wrap.setAttribute("data-rhythm-class", rhythmClass);
wrap.setAttribute("draggable", "true");
wrap.appendChild(scaleExerciseSvgClone(src, targetW));
exerciseBank.appendChild(wrap);
});
}

```

```

function playExerciseCompoundEighthEighthRest(token, freqHz) {
    stopPlayback();
    const beatMs = 60000 / bpm;
    const halfMs = beatMs / 2;
    ensureAudioContext();
    ensurePracticeOutGain();
    if (token !== exerciseSequenceToken) {
        releaseExercisePlayButtonAfterPlayback();
        return;
    }
    isMetronomeRunning = true;
    flashElement(bpmFlashBar);
    playMetronomeClick();
    triggerRhythmBarSustain(beatMs);
    armPracticeOutput();
    playQuarterNoteTone(halfMs / 1000, freqHz);
    if (playbackEndTimerId !== null) {
        window.clearTimeout(playbackEndTimerId);
        playbackEndTimerId = null;
    }
}

```

```

    playbackEndTimerId = window.setTimeout(function () {
        playbackEndTimerId = null;
        stopPlayback();
        if (token === exerciseSequenceToken) {
            releaseExercisePlayButtonAfterPlayback();
        }
    }, beatMs + 80);
}

exercisesPanel.addEventListener("dragstart", function (ev) {
    const t = ev.target.closest(".rhythm-drag");
    if (!t) {
        return;
    }
    draggedExerciseChip = t;
    ev.dataTransfer.effectAllowed = "move";
    ev.dataTransfer.setData(
        "text/plain",
        t.getAttribute("data-rhythm-class") || ""
    );
});
exercisesPanel.addEventListener("dragend", function () {
    draggedExerciseChip = null;
    refreshSlotFilledClass();
});

exerciseBank.addEventListener("dragover", function (ev) {
    ev.preventDefault();
    ev.dataTransfer.dropEffect = "move";
});
exerciseBank.addEventListener("drop", function (ev) {
    ev.preventDefault();
    const chip = draggedExerciseChip;
    if (!chip || !chip.classList.contains("rhythm-drag")) {
        return;
    }
    if (chip.parentElement === exerciseBank) {
        return;
    }
    chip.remove();
    refreshSlotFilledClass();
});

exerciseSlotsEl.addEventListener("dragover", function (ev) {

```

```

const zone = ev.target.closest(".drop-zone");
if (!zone) {
  return;
}
ev.preventDefault();
ev.dataTransfer.dropEffect = "move";
exerciseSlotsEl.querySelectorAll(".drop-zone").forEach(function (z) {
  z.classList.remove("drop-hover");
});
zone.classList.add("drop-hover");
});
exerciseSlotsEl.addEventListener("dragleave", function (ev) {
  const zone = ev.target.closest(".drop-zone");
  if (!zone) {
    return;
  }
  if (!zone.contains(ev.relatedTarget)) {
    zone.classList.remove("drop-hover");
  }
});
exerciseSlotsEl.addEventListener("drop", function (ev) {
  const zone = ev.target.closest(".drop-zone");
  if (!zone) {
    return;
  }
  ev.preventDefault();
  zone.classList.remove("drop-hover");
  const chip = draggedExerciseChip;
  if (!chip || !chip.classList.contains("rhythm-drag")) {
    return;
  }
  const fromBank = chip.parentElement === exerciseBank;
  const existing = zone.querySelector(".rhythm-drag");
  if (fromBank) {
    const clone = chip.cloneNode(true);
    clone.setAttribute("draggable", "true");
    if (existing) {
      existing.remove();
    }
    zone.appendChild(clone);
  } else {
    if (existing && existing !== chip) {
      const chipParent = chip.parentElement;
      if (chipParent && chipParent.classList.contains("drop-zone")) {

```

```

        chipParent.appendChild(existing);
    } else {
        existing.remove();
    }
}
zone.appendChild(chip);
}
refreshSlotFilledClass();
});

```

```

function playExerciseSequence() {
    const cfg = currentExerciseSpec();
    if (!cfg) {
        return;
    }
    if (exerciseReplayDisallowed) {
        return;
    }
    exerciseAnswerArea.hidden = false;
    const token = ++exerciseSequenceToken;
    exercisePlayBtn.disabled = true;
    exerciseFeedback.textContent = "";
    const freqHz = lessonPitchHz();

    if (cfg.playKind === "compound-eighth-eighth-rest") {
        playExerciseCompoundEighthEighthRest(token, freqHz);
        return;
    }
}

```

```

const seq = currentExerciseSequence() || cfg.sequence;
ensureAudioContext();
let i = 0;

```

```

function exerciseBeatWindowMs(rhythmClass) {
    if (rhythmClass === "eighth-note") {
        return (60000 / bpm) * 0.5;
    }
    if (rhythmClass === "sixteenth-note") {
        return (60000 / bpm) * 0.25;
    }
    if (rhythmClass === "sixteenth-rest") {
        return (60000 / bpm) * 0.25;
    }
    if (rhythmClass === "dotted-eighth") {

```

```

    return (60000 / bpm) * 0.75;
}
return 60000 / bpm;
}

function step() {
  if (token !== exerciseSequenceToken) {
    releaseExercisePlayButtonAfterPlayback();
    return;
  }
  if (i >= seq.length) {
    releaseExercisePlayButtonAfterPlayback();
    return;
  }
  const rc = seq[i];
  const beatOpts = { freqHz: freqHz };
  if (rc === "eighth-note") {
    beatOpts.beatFraction = 0.5;
  } else if (rc === "sixteenth-note") {
    beatOpts.beatFraction = 0.25;
  } else if (rc === "sixteenth-rest") {
    beatOpts.beatFraction = 0.25;
  } else if (rc === "dotted-eighth") {
    beatOpts.beatFraction = 0.75;
  }
  const stepBeatMs = exerciseBeatWindowMs(rc);
  playOneBeatForClass(rc, beatOpts);
  window.setTimeout(function () {
    if (token !== exerciseSequenceToken) {
      releaseExercisePlayButtonAfterPlayback();
      return;
    }
    i += 1;
    step();
  }, stepBeatMs);
}

step();
}

function exerciseSeriesHasMoreSteps() {
  const series = EXERCISE_CONFIG[exerciseSourceClass];
  if (!series || !series.exercises) {
    return false;
  }

```



```

    }
    if (series.infinite) {
        return true;
    }
    return exerciseStepIndex + 1 < series.exercises.length;
}

```

```

function revealExerciseNextNav() {
    const moreInSeries = exerciseSeriesHasMoreSteps();
    exerciseNextBtn.textContent = moreInSeries ? "Next exercise" : "Next";
    exerciseNextBtn.setAttribute(
        "aria-label",
        moreInSeries ? "Next exercise" : "Next rhythm lesson"
    );
    exerciseNextBtn.hidden = false;
}

```

```

exercisePlayBtn.addEventListener("click", function () {
    if (exerciseReplayDisallowed) {
        return;
    }
    playExerciseSequence();
});

```

```

exerciseCheckBtn.addEventListener("click", function () {
    const cfg = currentExerciseSpec();
    if (!cfg) {
        return;
    }
    const seq = currentExerciseSequence() || cfg.sequence;
    let ok = true;
    for (let s = 0; s < seq.length; s += 1) {
        const zone = exerciseSlotsEl.querySelector(
            '.drop-zone[data-slot="' + s + '"]'
        );
        const chip = zone && zone.querySelector(".rhythm-drag");
        const got = chip ? chip.getAttribute("data-rhythm-class") : "";
        if (got !== seq[s]) {
            ok = false;
            break;
        }
    }
    if (!ok) {
        exerciseGroupTotalChecks += 1;
    }
}

```

```

exerciseReplayDisallowed = true;
exercisePlayBtn.disabled = true;
clearExerciseSlotContents();
exerciseAnswerArea.hidden = true;
exerciseFeedback.textContent =
    "Not quite — you can't hear that sequence again. Press Next to continue.";
revealExerciseNextNav();
return;
}
exerciseGroupTotalChecks += 1;
exerciseGroupCorrectChecks += 1;
const moreInSeries = exerciseSeriesHasMoreSteps();
exerciseFeedback.textContent =
    cfg.successMessage +
    (moreInSeries
        ? " Press Next for the next exercise."
        : " Press Next to open the next rhythm lesson.");
revealExerciseNextNav();
});

exerciseNextBtn.addEventListener("click", function () {
    exerciseSequenceToken += 1;
    stopPlayback();
    exerciseNextBtn.hidden = true;
    const series = EXERCISE_CONFIG[exerciseSourceClass];
    if (!series || !series.exercises || !series.exercises.length) {
        return;
    }
    if (series.infinite) {
        exerciseStepIndex = (exerciseStepIndex + 1) % series.exercises.length;
        exerciseGroupTotalChecks = 0;
        exerciseGroupCorrectChecks = 0;
        if (
            series.groupRandomConfig &&
            exerciseStepIndex === 0
        ) {
            exerciseGroupSequences = generateGroupSequences(
                series.groupRandomConfig,
                series.exercises.length
            );
        }
        setupExerciseStep();
        return;
    }
});

```

```

    if (exerciseStepIndex + 1 < series.exercises.length) {
        exerciseStepIndex += 1;
        setupExerciseStep();
        return;
    }
    exerciseStepIndex = 0;
    const total = exerciseGroupTotalChecks;
    const correct = exerciseGroupCorrectChecks;
    const passedGroup = total > 0 && correct / total >= 0.8;
    if (!passedGroup) {
        exerciseNextBtn.hidden = true;
        showExerciseGroupFailScreen();
        return;
    }
    advanceToNextRhythmLessonFromStrip(exerciseSourceClass);
});

```

```

exerciseFailNextBtn.addEventListener("click", function () {
    exerciseSequenceToken += 1;
    stopPlayback();
    exerciseGroupTotalChecks = 0;
    exerciseGroupCorrectChecks = 0;
    hideExerciseGroupFailScreen();
    exercisesPanel.hidden = true;
    rhythmSession.classList.remove("exercises-mode");
    const tile = document.querySelector(
        ".rhythm-scroll-inner svg." + exerciseSourceClass
    );
    if (tile) {
        openRhythmLessonFromTile(tile);
    } else {
        rhythmSession.hidden = true;
        practicePanel.hidden = false;
    }
});

```

```

function initExercisesPanel() {
    if (!EXERCISE_CONFIG[exerciseSourceClass]) {
        exerciseSourceClass = "quarter-note";
    }
    exerciseStepIndex = 0;
    exerciseNextBtn.hidden = true;
    exerciseGroupTotalChecks = 0;
    exerciseGroupCorrectChecks = 0;

```

```

hideExerciseGroupFailScreen();
renderBpm();
rhythmSession.classList.add("exercises-mode");
const entry = EXERCISE_CONFIG[exerciseSourceClass];
if (
  entry &&
  entry.groupRandomConfig &&
  entry.exercises &&
  entry.exercises.length
) {
  exerciseGroupSequences = generateGroupSequences(
    entry.groupRandomConfig,
    entry.exercises.length
  );
} else {
  exerciseGroupSequences = null;
}
setupExerciseStep();
}

```

```

exerciseBackBtn.addEventListener("click", function () {
  exerciseSequenceToken += 1;
  stopPlayback();
  exercisesPanel.hidden = true;
  practicePanel.hidden = false;
  rhythmSession.hidden = true;
  rhythmSession.classList.remove("exercises-mode");
  hideExerciseGroupFailScreen();
  hideRhythmTrainerModeButtons();
  exerciseStepIndex = 0;
  exerciseNextBtn.hidden = true;
  exerciseGroupTotalChecks = 0;
  exerciseGroupCorrectChecks = 0;
});

```

```

detailNextBtn.addEventListener("click", function () {
  const lessonClass = activeRhythmClass;
  const entry = EXERCISE_CONFIG[lessonClass];
  const hasExercises =
    entry && entry.exercises && entry.exercises.length > 0;
  exerciseSequenceToken += 1;
  stopPlayback();
  if (hasExercises) {
    detailPanel.classList.remove("know-all-detail");
  }
}

```

```

    rhythmSession.classList.remove("know-all-detail");
    exerciseSourceClass = lessonClass;
    detailPanel.hidden = true;
    practicePanel.hidden = true;
    exercisesPanel.hidden = false;
    initExercisesPanel();
} else {
    advanceToNextRhythmLessonFromStrip(lessonClass);
}
});

startButton.addEventListener("click", function () {
    startButton.hidden = true;
    practicePanel.hidden = false;
    document.body.classList.add("has-started");
});

document.querySelectorAll(".rhythm-scroll-inner svg").forEach(function (tile) {
    tile.addEventListener("click", function () {
        openRhythmLessonFromTile(tile);
    });
});

knowAllRhythmsBtn.addEventListener("click", function () {
    showRhythmTrainerLanding();
});

trainerModeOriginalBtn.addEventListener("click", function () {
    enterRhythmTrainerWithMode("original");
});
trainerModeChallengingBtn.addEventListener("click", function () {
    enterRhythmTrainerWithMode("challenging");
});

detailPlayButton.addEventListener("click", function () {
    if (!activeRhythmClass) {
        return;
    }
    playOneBeat();
    if (!activeSecondText) {
        return;
    }
    detailSecondText.textContent = activeSecondText;
    detailSecondText.hidden = false;

```

```
});  
  
detailStopButton.addEventListener("click", function () {  
    stopPlayback();  
});  
</script>  
</body>  
</html>
```